

Litter: Location-based Social Media

David McAlister

4th Year Project Report
Computer Science
School of Informatics
University of Edinburgh

2021

Abstract

Despite the abundance of social media and availability of location technology, currently no app or service successfully combines the two, leaving a gap in the market. The goal of this project is to develop a location-based social media platform that fully implements both of these ideas, operating on Android.

Acknowledgements

I am very grateful to have the opportunity to work on this project and I really do hope future students can pursue similar ones. Here I would like to thank a few of the many who helped the project:

Thank you Stephen for being an encouraging and pleasant supervisor, without your backing and guidance I would not enjoyed the project as much I did.

Thank you to the volunteers who gave up their time to test my project, providing my project a level of scrutiny I could never give.

Thank you to Jesus, who has shown me that through resits, failure, toil and stress, I can still be content. At many points I considered abandoning this degree.

And of course, special thanks to everyone who reads this - I hope it's not too dull.

Table of Contents

1	Introduction	1
2	Background	2
2.1	Location-Based Social Media	2
2.1.1	Defunct Apps	2
2.1.2	Current Competitors	2
2.1.3	Comparison to Litter	3
2.2	Location-Based Games	3
2.2.1	Geocaching	3
2.2.2	Ingress	4
2.2.3	Pokemon GO	4
2.2.4	Inspirations for Litter	4
3	Design	5
3.1	Requirements	5
3.2	Social Media Features	5
3.3	Location-based Features	7
3.4	Android	7
4	Decisions	8
4.1	Programming Language	8
4.1.1	C/C++	8
4.1.2	Java	8
4.1.3	Kotlin	9
4.2	Database System	9
4.2.1	SQLite/Room	9
4.2.2	MongoDB	9
4.2.3	Cloud Firestore	10
4.3	Geo-Location	10
4.4	Displaying Messages	10
4.5	Activities and Fragments	11
4.5.1	Authentication Activity	11
4.5.2	Main Activity	11
5	Implementation	12
5.1	Authentication	12

5.1.1	Storing Users Emails and Passwords	12
5.1.2	AuthenticationActivity	14
5.2	Location-tracking	16
5.2.1	MainActivity	16
5.3	Cloud Firestore Collections	17
5.3.1	User	18
5.3.2	Message	18
5.3.3	MessageState	18
5.3.4	Comment	19
5.4	Creating Messages	19
5.4.1	NewMessageFragment	20
5.4.2	Uploading Images	21
5.4.3	ReviewFragment	22
5.5	Viewing Messages	22
5.5.1	Displaying Markers on the Map	22
5.5.2	Viewing Messages via Markers	25
5.5.3	EditMessageFragment	25
5.6	Exploration and Proximity	25
5.7	Comments	26
5.8	MessagesFragment	28
5.9	Kept Messages	29
5.10	SettingsFragment	30
6	Evaluation	31
6.1	Research Questions	31
6.1.1	Usability	31
6.1.2	Social-Media	31
6.1.3	Location-based features	32
6.2	Methodology	32
6.2.1	Tasks	33
6.2.2	Discussion	33
6.2.3	Questionnaire	34
6.2.4	Participants	34
6.3	Results	34
6.3.1	Viewing Messages	34
6.3.2	Comments	36
6.3.3	Creating Messages	36
6.3.4	Keeping Messages	37
6.3.5	Exploration	37
6.3.6	Other Issues and Suggestions	37
6.4	Recommended Fixes and Changes	38
7	Conclusion	39
7.1	Discussion of Achievements	39
7.2	Future Work	39
7.3	Future Testing	40

Chapter 1

Introduction

Nowadays most smartphones have a built-in map, helping users easily place where they are in the world. This functionality is no longer rare and expensive, allowing many developers to find unique uses for the technology. Plenty of apps such as Waze[1], Spot Adventure[2] and Foursquare City Guide[3] occupy various niches of location-based services. However, there is still one location-based niche that is neglected: social media.

Most location-based apps focus on connecting activities to places, whereas Litter connects people to places - it is a location-based social media. I fully explore the location-based apps of past and present in chapter 2, looking for direct competitors and inspirations for Litter to improve upon.

Litter is similar to Twitter in both name and function, providing a platform for people to broadcast messages for others to interact with. However, Litter is distinct in the way users interact. Users interact with messages over a map, where each message is tagged with a location. In chapter 3 I fully discuss the goals, concepts and design of Litter for it to truly be both location-based and a social media app.

In chapter 4 I discuss how best to achieve this design, comparing and contrasting my options. This is where I justify the technical, aesthetic and functional decisions which would help me reach my goals.

Litter required extensive coding in order to look and operate as a social media app. In chapter 5 I detail how exactly the design was implemented, explaining how each feature is programmed, showing examples from the code when helpful.

In order to gain insight into how well the objectives are met, I tested Litter on a group of potential users. In chapter 6 I show how they evaluated the design, what the results were and what they mean.

Finally, in chapter 7 I draw the conclusion that Litter does meet the definition of a location-based social media, while also highlighting areas for future improvement.

Chapter 2

Background

2.1 Location-Based Social Media

While Litter is a new product, it is not the first to combine social media and location features.

2.1.1 Defunct Apps

While currently no app claims the title of "Location-based Twitter", claimants did exist. Blummi was one example, where its creators called it "the world's first location-based social network based on Twitter".[4] The site no longer exists, yet archives of it go back to 20th August 2008, telling us the app was in private beta. In early 2012 the site went down and no further information on the project was published. As no copies are available, it's hard to say what functionality it had.

Other applications decided to instead integrate their services with Twitter, rather than provide complete and independent functionality. EagleTweet[5], Twinkle[6] and Twibble[7] were all examples of this. These apps performed location-based tasks, relying upon Twitter for social media aspect of their platform, where users share information to Twitter from the other app. The majority of these location-based apps relied upon Yahoo's FireEagle API[8], which was closed in 2013.

2.1.2 Current Competitors

Despite the abandonment of many location-based social media apps, there are still location-related social platforms today. Foursquare Swarm was released in 2014 as a spin-off of Foursquare City Guide.[9][10] Users are encouraged to 'check-in' and record a memory at their current location which they can share and talk about with friends. It operates like a gamified Yelp[11], where users can compete for 'mayorship' of a restaurant or store to receive discounts and perks there.

SKOUT on other hand, is one of many dating apps, which has always had a focus on location.[12] Similar to Swarm, SKOUT has a 'check-in' feature allowing users to share their location and interact with others. In 2014 they released Fuse[13] which

experimented with the idea of temporary messages (similar to Snapchat). The idea was that a user would create a group which any nearby users could join and interact with. Ten minutes after creation the chat and its contents would disappear. It never caught on and although Fuse was shut down, SKOUT still survives to compete with other location-based dating apps.

2.1.3 Comparison to Litter

There are several immediate problems when comparing these apps to Litter. Firstly, the few apps which aim to fill the same niche are gone, with little record of them to show that they existed at all. Several of these apps (such as EagleTweet) sound more like extensions to Twitter, rather than a separate, distinct social media like Litter.

On the other hand, social location-based apps which do exist today differ fundamentally to Litter. Swarm is a spin-off of (and is built from features originally present in) Foursquare's City Guide, where locations have "memories" attached to them rather than reviews. Lifelogging, mayorships and challenges are its main features, with the major social interaction between users being competition to win these challenges and mayorships. While users can tag their photos and thoughts to locations to be shared and seen within Swarm, users can also share their "check-in" to Twitter or Facebook. Private messaging is possible in Swarm, yet in the past users could broadcast and chat to people nearby. At this point, Swarm behaves more like a social media with location-based features, rather than a location-based social media. Similarly, SKOUT is a dating app with the ability to "check-in" to a location and talk to people nearby. Note that if we call Swarm and SKOUT location-based social medias, we must also call Facebook, Twitter and Whatsapp location-based social medias, as these all provide options to share location.[14][15][16]

After examining the current condition of location-based social media, and the history of its development, there is nothing that directly competes with Litter. Yes, Litter is not the first location-based social media, but it is unique amongst its peers today. Later on we will examine how Litter fully distinguishes itself as both location-based and social media in chapter 3.

2.2 Location-Based Games

With a limited number of location-based social-media apps to improve upon, I looked elsewhere for ideas to help users engage and socialise via locations. With many location-based games today being widely popular and having strong social aspects, this was a key genre which influenced Litter's design.

2.2.1 Geocaching

Geocaching is a real-world treasure hunt for a hidden 'cache' using only GPS coordinates and clues.[17] These 'caches' contain puzzles, prizes and trinkets to reward the finder. Upon discovery the players sign and date their discovery in the cache's

logbook before posting their experience online. Websites for geocaching have existed since 2000, and several different mobile apps are also available.

The most popular is Geocaching,[18] which allows users to search, create and log geocaches. While free to download, users are limited to what they can see and do until they pay for all features. Another popular and free geocaching app is c:geo.[19] Despite geocaching being a fun activity to do with others, these apps do not provide a way for users to interact or communicate and so should not be thought of as a form of location-based social media.

2.2.2 Ingress

One of the earliest Augmented Reality (AR) mobile games is Ingress.[20] Instead of a completely virtual environment, Augmented Reality games operate using real-world objects and environments. Ingress creates sites shown to players on a real-world map via mobile, which must be travelled to by players in order to interact with. Ingress differs from geocaching as individuals work together in one of two teams, competing against the other. Players often coordinate on a large-scale, for example one event involved 200 players over four months linking Germany, Greece and Ukraine together for their faction.[21]

2.2.3 Pokemon GO

By far the most well known location-based game is Pokemon GO, made by the creators of Ingress.[22] Although originally an April Fools' Joke,[23] two years later Google and Nintendo released the location-based AR game for mobile devices. Similar to Ingress, players move to and from locations in order to catch Pokémon, trade or battle with other players. A global success, Pokémon GO has been praised for being a "social-media phenomenon" that brings people together.[24][25]

2.2.4 Inspirations for Litter

Although Litter is location-based social media and not a game, there will be some location-based aspects of it, inspired by Geocaching and AR games like Pokémon GO.

Firstly, I want users to move in order to interact with others, similar to Pokémon GO's gyms or Ingress' AR sites. The location of a user should affect what the user sees.

Secondly, I want to encourage users to explore and search locations for interactions, finding a reward similar to Geocaching.

These two ideas are meant to ground Litter as a location-based social media, instead of a social media with location-based features.

Chapter 3

Design

3.1 Requirements

Each feature of Litter is designed to meet one of the three basic requirements:

1. Litter must be a *Social Media*,
2. Litter must be *Location-based*,
3. Litter must be an *Android App*.

3.2 Social Media Features

Firstly, Litter is a social media platform allowing users to see and interact with their own and others' messages. Figure 3.1 shows mockups of creating and viewing messages.

Each message has a short and unique title which acts as the main headline displayed on the map. The larger bulk of the message is below, in the center. Users can hide their identity from others and post anonymously, as messages display the username of the author. Each message has a view count which increments when new users see the message for the first time. Users also have the option to “keep” messages they like, saving them for later, akin to “liking” or “favouriting” the message. Users can also comment on messages, directly communicating with the author and other users.

Users also have the ability to visit messages from a list, similar to other social media. This provides an alternative way to access messages, which is difficult when several messages occupy and clutter a single location. Even when messages are far apart, it can be difficult to find a specific message out of several. Figure 3.2 shows a list of messages the user has kept, which the user can easily refer to and quickly scan for a particular message.

Finally, in order to provide a social media platform, data needs to be regularly stored and retrieved from an online database. Primarily, user data needs stored. For example,

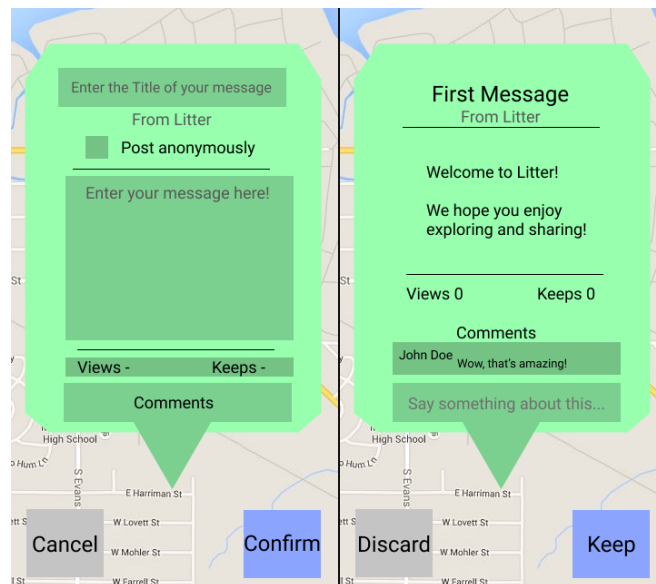


Figure 3.1: Mockups of what the user sees when creating a new message (left) and viewing a posted message (right).

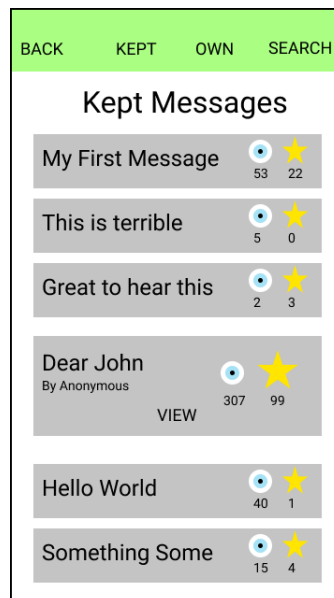


Figure 3.2: A mockup showing a list of all messages the user has kept.

users must be authenticated before they can interact with messages and other users. Thus a collection of users and their details, including emails and passwords, need stored to allow users to be create an account and login to Litter. Another example is messages. Whenever a user adds (or updates) a message, this message (and its details) should be accurate and available for all users to view and interact with. Similarly, when a message is deleted from the database, that message should also be removed for other users. Other details such as usernames, view counts, comments and which messages users have seen also need to be tracked in a database.

3.3 Location-based Features

Secondly, Litter is a location-based app hence every post and user must have a location. Users create and view messages at specific locations by moving around in the real world, which updates their position on the map. User location is tracked and messages display on a map for them to interact with.

Only by travelling to and exploring messages, can users communicate and interact with others. From far away users can see where a message is, but cannot read or interact with it. This achieves the first idea of section 2.2.4, as users need to move.

As the user gets closer, part of the message's title is “decoded”, revealing more details and encouraging them to travel closer. Only once they are in the immediate vicinity can the message be fully read and interacted with. This achieves the second idea of chapter 2.2.4, as users are rewarded for exploring.

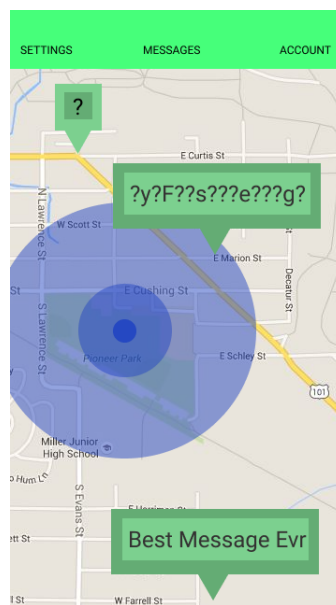


Figure 3.3: A mockup of the map showing a user near three messages, one hidden (top), one partially seen (top center) and the other already seen (bottom).

Figure 3.3 shows a user next to 3 messages. There are 3 blue circles of varying sizes: the smallest circle represents the user's current location, whereas the other two represent “ranges” at which the user can “decode” and view a message. One message is at the top where the title is hidden, one message is partially revealed (center) and one message at the bottom is already visible. As the center message is within the larger “decode” range, part of its title is visible, whereas the top message is outside and its title is unknown.

3.4 Android

The requirement that Litter be an Android app does not directly affect *what* features are present, but rather impacts *how* these features are implemented.

Chapter 4

Decisions

As part of the process of creating Litter, several comparisons were made to help decide the best way to implement Litter.

4.1 Programming Language

Out of several options, C/C++, Java and Kotlin are the most common programming languages which can be used to create Android apps.

4.1.1 C/C++

While not as heavily supported, C and C++ can be used to code and run Android apps using NDK and its libraries.

The official guide[26] states two advantages to using C/C++:

- *“Squeeze extra performance out of a device to achieve low latency or run computationally intensive applications, such as games or physics simulations.”*
- *“Reuse your own or other developers’ C or C++ libraries.”*

To begin with, Litter is a location-based social media, there is little need for any extra high performance and efficiency. Regarding the second point, there are other (and also more!) specialist Android app libraries provided for Kotlin or Java compared to C/C++.

4.1.2 Java

Java is another choice, used today in many apps. Despite this, Android development is encouraged towards Kotlin, especially since Google is promoting a “Kotlin-first, Java second” approach.[27] For example, Twitter developers have said that coding in Kotlin is more productive and reliable than Java.[28]

4.1.3 Kotlin

There are several additional reasons why I chose to develop Litter using Kotlin instead of Java (or C/C++).

Firstly, Kotlin is widely supported, especially by Google. While not as old as Java or C++, Kotlin has a wealth of documentation and resources online which I can refer to. Secondly, Kotlin can be used interchangeably with Java code. Kotlin even runs on the Java Virtual Machine. As I am experienced in Java coding, many functions and methods I am familiar with in Java also run in Kotlin. Kotlin's interoperability with Java allows me to make use of my Java knowledge. Thirdly, Kotlin is more concise than and reduces the boilerplate in Java. For example, unlike Java, getters and setters in classes do not need to be set up in Kotlin. Kotlin also has various features such as lambda expressions and null safety which should reduce the amount of code needed. Finally, Kotlin is supported by AndroidStudio.[29] AndroidStudio is the main IDE (interactive development environment) for Android apps, having features such as an emulator and visual designer. These features will reduce the time needed to design, code and test Litter.

4.2 Database System

There are several database systems that would allow data to be saved and retrieved using an Android app.

4.2.1 SQLite/Room

One platform for managing storing and retrieving data is SQLite, a relational database management system.[30] While powerful to flexible to use, the official Android documentation describes SQLite as *"fairly low-level...[requiring] a great deal of time and effort to use"*. [31] In particular, the need to manually update SQL queries is *"time-consuming and error-prone"*, and *"lots of boilerplate code to convert between SQL queries and data objects"* is required. Android Jetpack's Room does solve and reduce these problems, yet the documentation suggests that Room (and SQLite) is intended for offline, local databases.[32] Litter needs to frequently check and update the database according to user interactions. To do this a web server to store the data would also need to be setup.

4.2.2 MongoDB

MongoDB is a NoSQL, document-based, distributed database hosted on the cloud.[33] For Android, the MongoDB Realm Android SDK is used to connect to a MongoDB Atlas database. MongoDB has great performance and flexibility, but similarly to my analysis of C/C++, this extra performance is unnecessary for a social-media app.

4.2.3 Cloud Firestore

Google Firebase's Cloud Firestore is another cloud-based, NoSQL database system similar to MongoDB.[34] There are several reasons why I decided to use it over MongoDB. As it is provided by Google, Cloud Firestore is well supported with plenty of tutorials and resources, catered to ease integration with Kotlin and Android. In contrast, documentation and resources that cover MongoDB integration with Android are considerably more scarce compared to Firebase. Even MongoDB concedes that Firebase *"has more features for mobile applications"* and is *"designed in particular for mobile app developers"*.[35] Firebase's boilerplate for several key features reduces time and code needed to setup file/image hosting, authentication and hosting, which is a considerable advantage over MongoDB.

4.3 Geo-Location

Litter needs to place users and posts on a real-world map, requiring geo-location functionality. While possible to create and implement myself, it would take far too long. There is no point reinventing the wheel, especially when pre-built APIs such as Mapbox[36] and Google Maps[37] handle geo-location services well. Both APIs are location-services platforms, which provide mapping functionality to Android apps. The level of documentation for MapBox is reasonable, but less than Google Maps, which is a point in Google Maps' favour. A serious issue of MapBox is its poor location coverage and accuracy. MapBox relies on (mostly) free data in order to run mapping services and as such is much more inconsistent compared to Google Maps. One possible advantage of MapBox compared to Google Maps is that MapBox's services are lower priced. However, Google Maps provides limited services on Android apps for free, which are adequate for Litter. As Google Maps provides better accuracy, documentation and services for free I decided to use it to implement Litter.

4.4 Displaying Messages

Although the format and layout of all messages are the same, there is one question which determines how a user interacts with it: is the current user viewing the message also the author of that message? Authors should the power and freedom to update and delete their message, but not the ability to keep it. On the other hand, non-authors have the ability to keep a message, but not alter it. In order to allow users to quickly identify and distinguish their own messages, I decided to colour own messages as blue, whereas other messages are green. Messages created by other users also have different states - a message can be in a seen, decoded, unseen or kept state depending on where the user is and how the user behaves. However, as the message's title is decoded based on how close the user is these messages are all green, with the exception that kept messages are orange. Instead, the visibility of title differs depending on how close the user is. Initially, a message is "unseen" and the user sees question marks instead of a title. As the message moves into the user's decode range, half the title becomes visible. Once the user gets close the title is fully revealed and the message can be viewed. Another

decision was to display a title to mark a message's location on the map, rather than just a coloured icon. This allows users to see and identify distinct messages quickly, even when several share the same type (same author) or state (seen/decoded/unseen).

4.5 Activities and Fragments

Another decision was how to structure the UI and layout of Litter. I decided to separate Litter into two distinct “flows”, each with their own Activity and several Fragments. An Activity can be thought of as a screen or window where the UI is displayed,^[38] whereas a Fragment is modular, smaller layout running within an Activity.^[39]

4.5.1 Authentication Activity

The Authentication Activity is where a user performs authentication tasks, i.e. creating and logging in to their account. Within this activity are three fragments each dealing with a separate UI and task: Register (or Create) an Account, Reset Password and Login. Rather than start and stop different activities, which can be computationally intensive, or dynamically alter the visibility of groups of UI objects, which complicates code and clutters the layout, fragments are swapped in and out as needed. Splitting the code for authentication three ways also improves the readability.

4.5.2 Main Activity

The Main Activity is where users interact with the map and messages. Fragments are needed to display the Map, list of all Messages, Create a message, View a message and Edit/Delete a message. The three most important fragments are displayed as tabs across the top of the activity: Map, (list of all) Messages and Settings. This ensures quick and direct access from within the activity. Creating a message is done via a floating action button on the map. Messages can only be viewed when their map icon is clicked on and only when a user is close enough. If a user owns a message clicking on it swaps in the Edit Fragment. While possible to group some fragments into their own activity, i.e a *Message Activity*, there are some problems with this. First, messages are displayed on top of the map and as such still run within the Main Activity. Second, messages are opened and closed frequently from the map, and using another activity requires extra code to communicate and computation time to load each layout. Additionally, a Settings fragment is needed to allow the user to log out, display statistics and allow users to hide different map objects. I decided to incorporate statistics as a way for users to measure their performance in exploring, providing more incentive to go out, find and create messages. Adding the ability to filter out or hide different map objects provides a way to simplify the map. For example, it allows a user to hide their own messages (or messages they've already seen) so they can easily see which messages are unexplored. Similarly to showing all messages in a list, it also allows users to better view messages that are obstructed by other messages.

Chapter 5

Implementation

5.1 Authentication

5.1.1 Storing Users Emails and Passwords

Section 3.2 explains that user details are stored in order to allow users to be authenticated (i.e. users must create accounts and login to Litter before interacting with others) and section 4.2.3 explains why Cloud Firestore will be used to store this user data. This section covers how Cloud Firestore was used to implement user authentication.

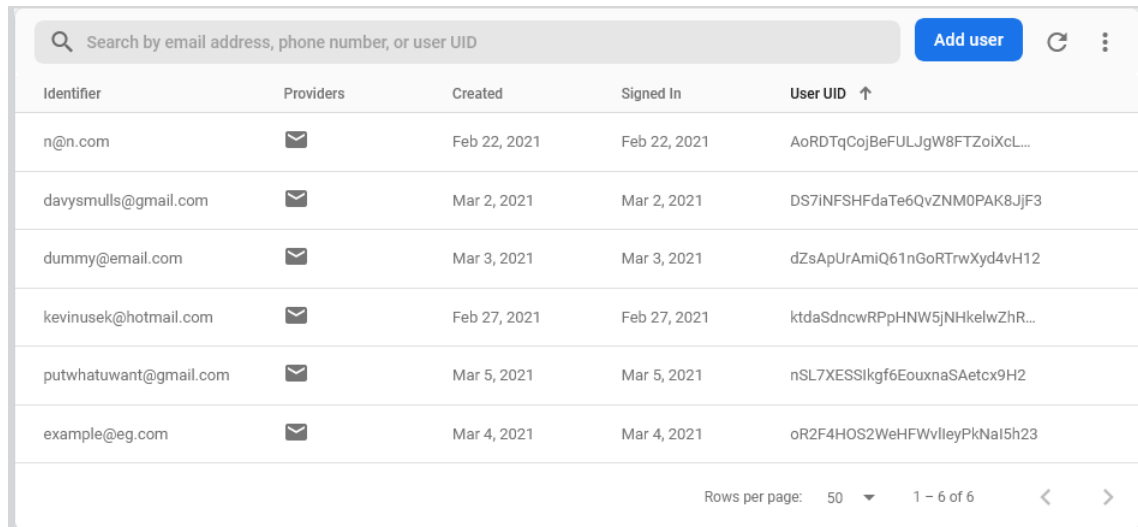
Cloud Firestore offers several boilerplate features for authentication, the primary one I used being *Email and password based authentication*.^[40] First the security rules had to be defined to allow recognised users to sign in.^[41] Litter users must be able to access other documents (i.e. messages and the users that own them) in order to view messages and the authors that own them. Therefore, I avoided restrictions or rules which could conflict with these features, instead going only with the simple rule that a user must be authenticated (i.e. valid, matching email and password), seen in Figure 5.1.

```
1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4     match /{document=**} {
5       allow read, write: if request.auth.uid != null
6     }
7   }
8 }
```

Figure 5.1: These security rules permit users to log in and access data only if they have a valid request (correct email and password).

The following steps I took are outlined in the official documentation.^[42] Next, Firebase and its dependencies are added to the Android project. For each fragment or activity that requires access to the details of the currently logged in user (or soon to be logged in), a `FirebaseAuth` variable is declared and initialised. This `FirebaseAuth`

object is used to perform various actions such as account creation or login. The implementation of these actions are covered in section 5.1.2. Whenever a user is created, their email, password and User ID (UID) is stored in a secure database outside of Cloud Firestore (Figure 5.2).

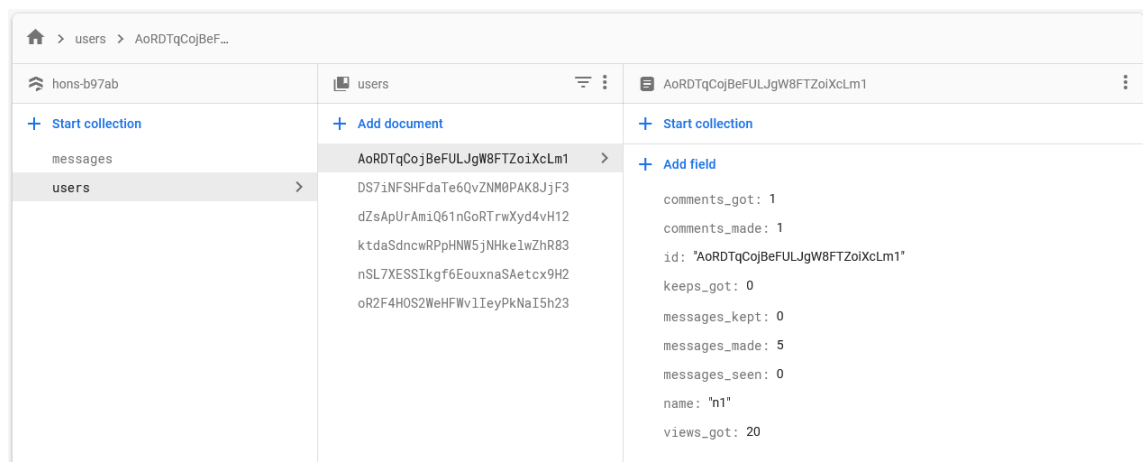


Identifier	Providers	Created	Signed In	User UID ↑
n@n.com	📧	Feb 22, 2021	Feb 22, 2021	AoRDTqCojBeFULJgW8FTZoiXcLm1
davysmulls@gmail.com	📧	Mar 2, 2021	Mar 2, 2021	DS7iNFSHFdaTe6QvZNM0PAK8JjF3
dummy@email.com	📧	Mar 3, 2021	Mar 3, 2021	dZsApUrAmiQ61nGoRTrwXyd4vH12
kevinusek@hotmail.com	📧	Feb 27, 2021	Feb 27, 2021	ktDaSdncwRPpHNW5jNHkelwZhr83
putwhatuwant@gmail.com	📧	Mar 5, 2021	Mar 5, 2021	nSL7XESSIkGf6EouxnaSAetcx9H2
example@eg.com	📧	Mar 4, 2021	Mar 4, 2021	oR2F4HOS2WeHFWvIleyPkNaI5h23

Rows per page: 50 1 – 6 of 6

Figure 5.2: When a user registers an account, their email, passwords and UID are stored here using Firebase's Authentication service.

However, several other user details that need stored cannot be stored here. For example, a message's author should not display their email (for security/privacy reasons) but rather their username - which needs to be stored and retrieved from somewhere! To do this, a `users` collection is stored on Cloud Firestore (Figure 5.3). Every time a user creates an account, a new entry is also created in the `users` collection with a matching UID. Here various details such as the user's name are saved - these details are covered in full in section 5.3.1.



Collection	Document	Fields
messages		
users	AoRDTqCojBeFULJgW8FTZoiXcLm1	comments_got: 1 comments_made: 1 id: "AoRDTqCojBeFULJgW8FTZoiXcLm1" keeps_got: 0 messages_kept: 0 messages_made: 5 messages_seen: 0 name: "n1" views_got: 20

Figure 5.3: Although the `users` collection contains further details irrelevant to authentication, each entry needs to be linked to the authentication database with a matching UID.

```

1  override fun onCreate(savedInstanceState: Bundle?) {
2      super.onCreate(savedInstanceState)
3      setContentView(R.layout.activity_authentication) //Load AuthenticationActivity Layout and UI
4      backButton = findViewById(R.id.buttonBackAuth) //Find the back button
5      backButton.visibility = View.INVISIBLE //Set back button in AppBar to be invisible
6      backButton.setOnClickListener(this) //But it will listen for clicks (when visible)
7      supportFragmentManager.beginTransaction().apply { //Call the fragment manager to prepare swap
8          replace(R.id.frameLayoutAuth, LoginFragment()) //Prepare to replace FrameLayout's contents with LoginFragment
9          setTransition(FragmentTransaction.TRANSIT_FRAGMENT_FADE) //Set the animation of the swap to be a fade
10         commit() //Make the swap!
11     }
12 }

```

Figure 5.4: Code from `AuthenticationActivity` showing how `LoginFragment` is displayed within a `FrameLayout`

5.1.2 AuthenticationActivity

As specified in section 4.5.1, the `AuthenticationActivity` class is used to perform various activities such as logging in or creating an account.

5.1.2.1 Swapping Fragments

As a result of using Fragments instead of Activities (section 4.5), Litter needs a way to regularly load and unload Fragments smoothly. This “swapping” is done using the *FragmentManager* to replace the current contents of a layout.^[43] For example, `AuthenticationActivity`’s layout file `activity_authentication.xml` uses a `RelativeLayout` containing two components: an `AppBarLayout` with a `Button` and a `FrameLayout`, so `AuthenticationActivity` can dynamically display fragments based on the user’s actions (Figure 5.4-5.5). This ensures the user is greeted by a screen telling them to login, or sees the `RegisterFragment` when they click the *Create an account* text, and so on.

5.1.2.2 Logging In

In order to authenticate users Firebase’s `auth` object is used (Figure 5.6). Firstly, logging in uses the `signInWithEmailAndPassword` method with an `email` and `password` parameter.^[42] Although users are authenticated, further code is needed to ensure users understand what happens if their login failed, avoiding frustration and confusion. For example, Litter first checks the parameters before attempting to authenticate, alerting users via a dialog box why their login failed. Only then does Litter carry out and wait for the result of the authentication request before performing the corresponding action.

5.1.2.3 Skipping Authentication

When a user is authenticated once, there is no reason to ask them to authenticate again while they are still logged in. Instead, when a `LoginFragment` is loaded a check is made to see if a user has already logged in. If the user has logged in, they are sent immediately to `MainActivity`.

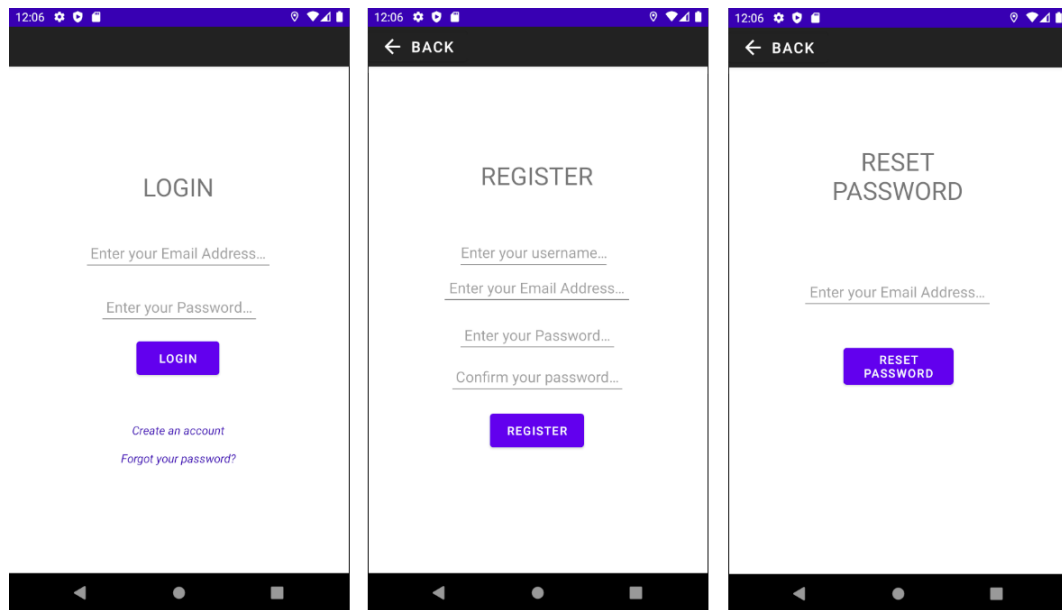


Figure 5.5: *AuthenticationActivity* when it contains a *LoginFragment* (left), *RegisterFragment* (middle) or *ResetFragment* (right).

```

1 private lateinit var auth: FirebaseAuth
2 auth = FirebaseAuth.getInstance()
3
4 auth.signInWithEmailAndPassword(email, password).addOnCompleteListener(this) { task ->
5     if (task.isSuccessful) { //If successfully signed in
6         startActivity() //start Main Activity
7     }
8 }
9
10 auth.createUserWithEmailAndPassword(email, password).addOnCompleteListener(this) { task ->
11     if (task.isSuccessful) { //If successfully created new user
12         db.collection("users").document(auth.uid!!).set(User(auth.uid!!, username)) //create user in Firestore
13     }
14 }
15
16 auth.sendPasswordResetEmail(email).addOnCompleteListener(this) { task ->
17     if (task.isSuccessful) { //if successfully sent password reset email
18         showResetPasswordDialog() //inform user to check their email
19     }
20 }

```

Figure 5.6: Code showing how authentication uses the *auth* object.

5.1.2.4 Creating an Account

Once again, creating/registering an account involves *auth*, calling the *createUserWithEmailAndPassword* function (Figure 5.6). As mentioned in section 5.1.1, there is also a need to create a *users* collection to store other details, such as username. Thus each time a new email and password account is created, a user document with the same UID also needs created in the *users* collection in the Cloud Firestore database. To do this a Firestore instance is initialised[44], and a *User* object (outlined in section 5.3.1) is set as a document (Figure 5.7).[45]

```

1 private val db: FirebaseFirestore = FirebaseFirestore.getInstance() //Get instance of Litter's Firestore database
2
3 db.collection("users") //In the users collection
4 .document(auth.uid!!) //Get a document using the UID as the key
5 .set(User(auth.uid!!, username)) //Set the contents of that document to a User object
6
7 db.collection("messages") //In the messages collection
8 .document(title) //Get a document using the title as the key
9 .update("image", "") //overwrite the image field to be ""
10 .update("text", new_message) //overwrite the text field to be new_message
11
12 db.collection("messages") //In the messages collection
13 .document(title).delete() //Delete the document with a key equal to title

```

Figure 5.7: Code from used to add, update or delete documents in the messages or users collection.

5.1.2.5 Resetting a Password

Similarly, resetting passwords is done using the `auth.sendPasswordResetEmail(email)` method (Figure 5.6).

5.2 Location-tracking

A vital step in meeting the location-based design of section 3.3 is location tracking. Each user needs to see their location in the world as they move around. This was done using Google Maps' services, as mentioned in section 4.3.

5.2.1 MainActivity

Although `MainActivity` performs several functions (as outlined in section 4.5.2), its primary function is to provide the map the user will see and interact with. Similar to `AuthenticationActivity`, `MainActivity` acts as a “fragment holder”, displaying different fragments depending on the user's interactions (Figure 5.8). Instead of using a `FrameLayout`, a `ViewPager`[46] below a `TabLayout` is used, where tapping a tab updates the `ViewPager`'s contents.[47]

5.2.1.1 Map

By default, the first fragment held in `MainActivity`'s `ViewPager` is `MapFragment`. `MapFragment` is used to display the map, track user location and show markers for each message.[48]

5.2.1.2 Location

Before a user's location can be retrieved, the user must grant Litter permission to access their device's location. To do this the permissions are requested manually, [49][50][51] as seen in Figure 5.9. If permission is denied, it is merely requested again as Litter cannot operate without it. Once permission is granted, the device's location

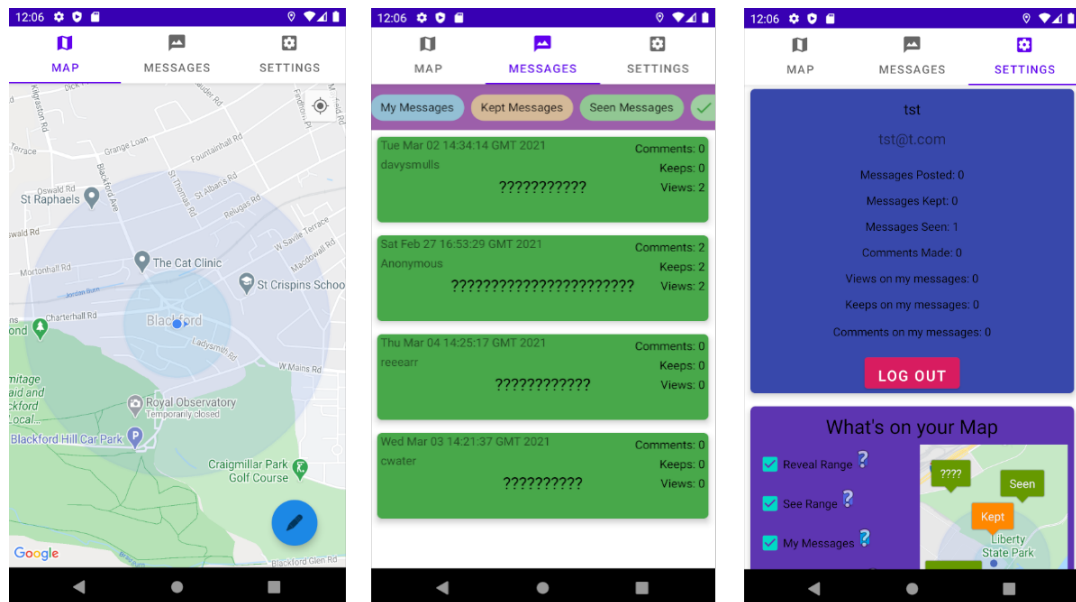


Figure 5.8: *MainActivity* when it contains a *MapFragment* (left), *MessagesFragment* (middle) or *SettingsFragment* (right).

is requested at regular intervals,[52] allowing the user's current location to be updated after each request.[53]

```

1 private fun getLocationPermission() {
2     if (ContextCompat.checkSelfPermission(this.requireContext(), Manifest.permission.ACCESS_FINE_LOCATION)
3         == PackageManager.PERMISSION_GRANTED) { //If we have permission to for high-accuracy location
4         locationPermissionGranted = true //We have permission!
5     } else { //Otherwise request permission for the fine-location services
6         ActivityCompat.requestPermissions(this.requireActivity(),
7             arrayOf(Manifest.permission.ACCESS_FINE_LOCATION), PERMISSIONS.REQUEST_ACCESS_FINE_LOCATION)
8     }
9 }
10
11 override fun onRequestPermissionsResult(requestCode: Int, permissions: Array<String>, grantResults: IntArray) {
12     super.onRequestPermissionsResult(requestCode, permissions, grantResults)
13     locationPermissionGranted = false
14     when (requestCode) {
15         PERMISSIONS.REQUEST_ACCESS_FINE_LOCATION -> { //If the request code is for high-accuracy location
16             if (grantResults.isNotEmpty() && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
17                 locationPermissionGranted = true //If it is granted we have permission!
18                 startLocationUpdates() //Start receiving location updates
19             }
20         }
21     }
22 }

```

Figure 5.9: Code from *MapFragment* used to confirm and request location permissions

5.3 Cloud Firestore Collections

As mentioned in section 4.2, data will be stored using Cloud Firestore. Cloud Firestore stores data in the fields of document objects, with many documents occupying a collection.[54] Litter has two Cloud Firestore collections. The first collection users

contains every registered user, with each document within corresponding to a unique user. If a user has been seen or explored any messages, that user will also contain a `seenmessages` collection, with each document corresponding to a message in the `messages` collection. The second collection `messages` contains every message every user has posted, with each document within corresponding to a unique message. If a message has been commented on, that message will also contain a `comments` collection, with each comment corresponding to a unique comment. In the following sections the document fields of each collection are explained.

5.3.1 User

The User object contains nine fields in total (Figure 5.3). Storing location is unnecessary as the location is retrieved locally from the device, never mind that constantly updating the database with the user's current location is expensive and time-consuming. Storing the user's email address was unnecessary as it is already stored via authentication. Figure 5.10 shows the data class and fields used to represent these documents.

```

1 data class User(
2     var id: String, //Unique identifier for a user, matches an Authentication account UID.
3     var name: String, //Used to identify a user as the author of a comment or message.
4     var messages.made: Int = 0, //Total number of messages the user has posted. Displayed in SettingFragment.
5     var messages.kept: Int = 0, //Total number of messages the user has kept. Displayed in SettingFragment.
6     var messages.seen: Int = 0, //Total number of messages the user has viewed. Displayed in SettingFragment.
7     var comments.made: Int = 0, //Total number of comments the user has made. Displayed in SettingFragment.
8     var views.got: Int = 0, //The total number of views all messages belonging to the user received.
9     //Displayed in SettingFragment.
10    var keeps.got: Int = 0, //The total number of keeps/saves on all messages belonging to the user received.
11    //Displayed in SettingFragment.
12    var comments.got: Int = 0 //The total number of comments made on all messages belonging to the user.
13    //Displayed in SettingFragment.
14 )

```

Figure 5.10: New documents in the `users` collection are created from and correspond to the `User` data class.

5.3.2 Message

The Message object contains ten fields in total (Figure 5.11). Figure 5.12 shows the data class and fields used to represent these documents.

5.3.3 MessageState

The Message object contains three fields in total (Figure 5.13). As mentioned in sections 3.3 and 4.4, a message can be in a seen, partially-seen or unseen state. This is represented by the `seen` field, a boolean. When `seen = true`, a message is fully seen and can be viewed, whereas `seen = false` indicates that a user has only somewhat decoded the message. I.e, to depend on how close a user is that a message, a `seenmessage` document either updates the `seen` field to true, or creates a new document with a `seen` value of false. Figure 5.14 shows how these documents are stored in Cloud Firestore, whereas Figure 5.15 shows the data class and fields used to represent these documents.

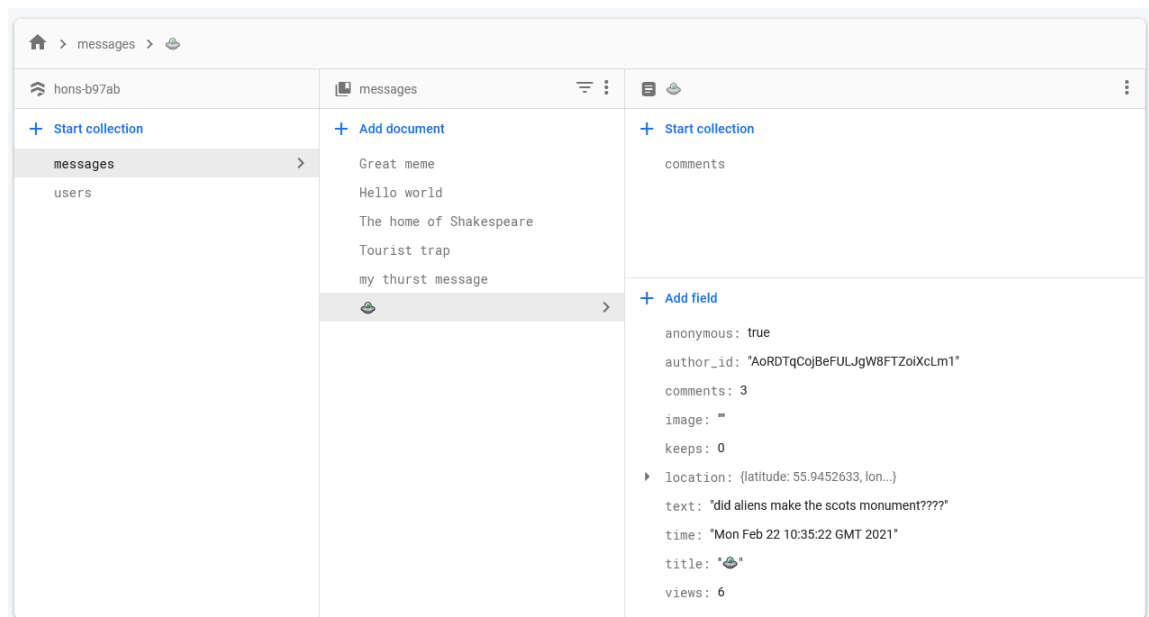


Figure 5.11: *The messages collection stores details of each message.*

```

1 data class Message(
2     var title : String? = null, //Unique identifier for a message, main headline of message, displayed on the map.
3     var author_id: String? = null, //UID of the author who made the message
4     var location : LatLng = LatLng(0.0,0.0), //Location of the message on the map.
5     var text : String? = null, //The text content of the message. Null when it message is an image message.
6     var image: String? = null, //The URI from where the image is downloaded. Null when it message is a text message.
7     var time: String = Timestamp.now().toDate().toString(), //Time and date message was originally posted.
8     var keeps: Int = 0, //number of keeps/saves received
9     var views: Int = 0, //number of views received
10    var comments: Int = 0, //number of comments made
11    var anonymous: Boolean = true //only if true is the author's username displayed
12 )

```

Figure 5.12: *New documents in the messages collection are created from and correspond to the Message data class.*

On the other hand if a user is too far away, a message is unseen and has no document in the seenmessages collection. This is done to avoid storing every message within every user's seenmessages collection, which vastly increases the database size.

5.3.4 Comment

The Comment object contains 4 fields in total (Figures 5.16-5.17).

5.4 Creating Messages

In order to create a message, first a user must tap the blue icon at the bottom right of their screen on the MapFragment (Figure 5.8). This hides MainActivity's ViewPager and TabLayout, making another FragmentContainer visible and placing a NewMessageFragment within. Initially it was intended for messages to be overlaid on top of the map (Figure

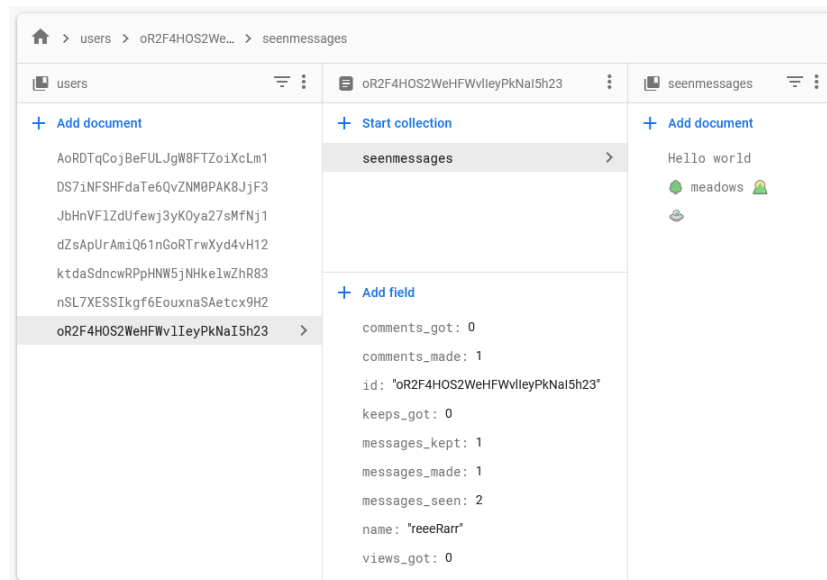


Figure 5.13: The *seenmessages* collection stores details of each message.

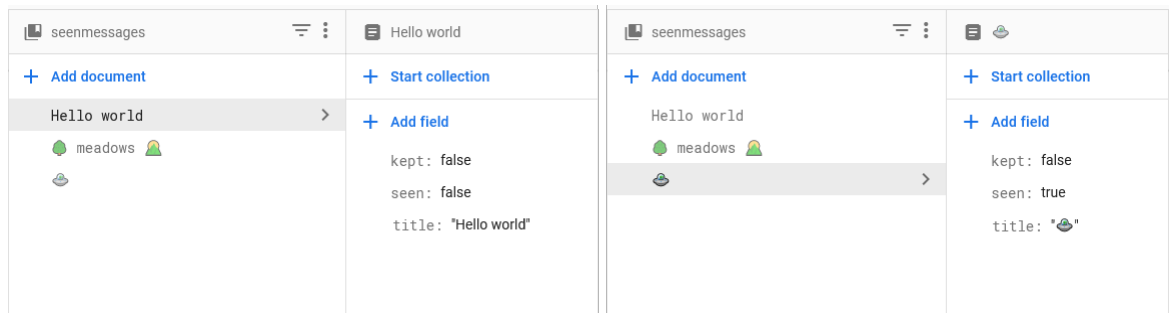


Figure 5.14: The *Hello World* message (left) has only been partially seen, whereas the *alien emoji* message (right) has been fully seen by the user.

```

1 data class MessageState(
2   var title : String, //Maps to a document in the "messages" collection.
3   var seen: Boolean = false, //True if message is fully-seen, false if only partially-seen,
4   //unseen messages are absent in collection.
5   var kept: Boolean = false, //True if the user has kept/saved this message.
6 )

```

Figure 5.15: New documents in the user's *seenmessages* collection are created from and correspond to the *MessageState* data class.

3.1), but the lack of screen space made reading messages difficult.

5.4.1 NewMessageFragment

Corresponding to a mockup given in Figure 3.1, the *NewMessageFragment* provides a form for the user to create a message (Figure 5.18). The content of a message can either be an image or a text message, but not both. This keeps messages simple and avoids an entire message being too big to fit on a screen at once. Even though a message's title

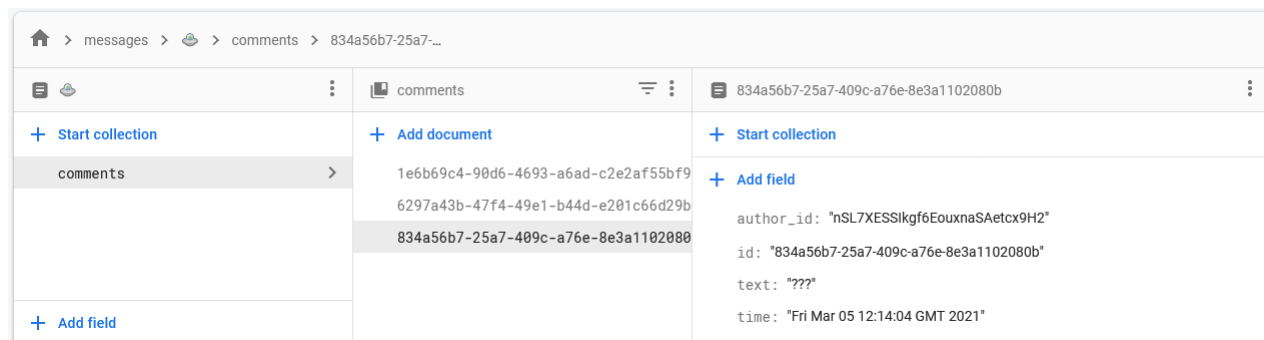


Figure 5.16: The *comments* collection stores details of each comment.

```

1 data class Comment(
2   val id: String? = null, //Unique identifier for a comment document.
3   val author.id: String? = null, //ID of author who made the comment.
4   val time: String? = null, //Time and date comment was posted.
5   val text: String? = null //Text content of comment.
6 )

```

Figure 5.17: New documents in a message's *comments* collection are created from and correspond to the *Comment* data class.

already exists as a brief description or headline, some users may have more to say, so the option to add more text instead of an image is available.

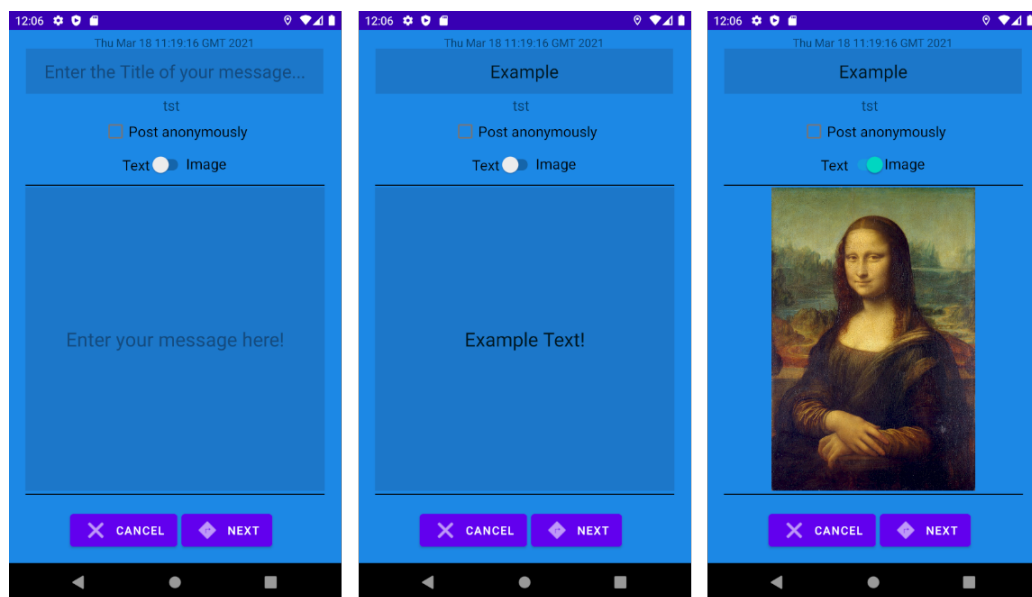


Figure 5.18: An empty *NewMessageFragment* (left), a *NewMessageFragment* with a text message (middle), and *NewMessageFragment* with an image (right).

5.4.2 Uploading Images

Allowing users to upload images was done using Firebase's Cloud Storage[55], whereby the user selects an image on their device (Figure 5.19) before the local file is uploaded

using the `putFile()` method (Figure 5.20).[56]

```

1 | R.id.imageViewMessage -> { //when image view clicked
2 |     val intent = Intent()
3 |     intent.type = "image/*"
4 |     intent.action = Intent.ACTION_GET_CONTENT
5 |     startActivityForResult(intent, 1) //start intent to get image from user
6 | }
7 |
8 | override fun onActivityResult(requestCode: Int, resultCode: Int, data: Intent?) {
9 |     super.onActivityResult(requestCode, resultCode, data)
10 |    if (requestCode == 1 && resultCode == RESULT_OK && data != null && data.data != null) { //if image is valid
11 |        imageMessage = data.data!! //save image data
12 |        imageView.setImageURI(imageMessage) //update imageview with image
13 |        tapImage.visibility = View.INVISIBLE //hide textview prompting user to add image
14 |    }
15 | }

```

Figure 5.19: Code from *NewMessageFragment* used to fetch an image from the user's device.

5.4.3 ReviewFragment

Before a new message is created, the user first reviews the location of the new message on the map - via *ReviewFragment* (Figure 5.21). This allows the user to confirm they are happy with the title and the location of the message, both of which cannot be edited after posting. Once the user confirms, the *Message* object is created from *NewMessageFragment*'s details and added to the Cloud Firestore *messages* collection (Figure 5.20).[45]

5.5 Viewing Messages

5.5.1 Displaying Markers on the Map

Rather than displaying entire messages on the map, messages first appear as named markers which must be clicked/tapped on by the user to view. Instead of using the default markers, messages instead appear as custom markers using Google Maps' *IconGenerator* class.[57][58] This allows text (i.e. message's title) to be displayed on the marker, uniquely identifying it, and for the icon's colour to be easily set (Figure 5.22).

Whenever a user's location is updated, so too are the map's markers. First, all messages from the *messages* collection are fetched. Only if the messages are nearby (within 10km) are they displayed on the map - this minimises the number of objects needed to be loaded and added to the map at one time, reducing latency and improving performance. As messages can only be viewed if the user is nearby, it is unnecessary to display messages which the user is unlikely to view where they are.

In order to track, store and display messages and their markers four *HashMap*s are used:

- The *markers* variable stores a mapping of a marker's ID to a *Marker* object (*HashMap<String, Marker>*), allowing markers on the map to be edited or

```

1 private lateinit var storage: FirebaseStorage
2 private lateinit var storageReference: StorageReference
3 storage = FirebaseStorage.getInstance()
4 storageReference = storage.reference
5
6 val randomKey : String = UUID.randomUUID().toString() //generate random key for image
7 val storageRef: StorageReference = storageReference.child("images/$randomKey") //generate unique URL to place image
8 if (messagetext == "") { //if it is not a text message (text is empty -> image message)
9     storageRef.putFile(image!!).addOnSuccessListener { taskSnapshot -> //if image successfully uploads
10         val message = Message( //Create a Message object to upload to Firestore
11             image = randomKey, //store the image key access image from storage
12             text = "", //leave the text field as null
13             title = titletext ,
14             location = LatLng(currentLocation.latitude , currentLocation.longitude),
15             author_id = auth.uid,
16             anonymous = anonymouspost
17         )
18         db.collection ("messages").document(titletext).set(message) //Add document to messages collection
19     }
20 }
21 else { //otherwise it is a text message
22     val message = Message(
23         image = "", //leave the image field as null
24         text = messagetext, //store the text message
25         title = titletext ,
26         location = LatLng(currentLocation.latitude , currentLocation.longitude),
27         author_id = auth.uid,
28         anonymous = anonymouspost
29     )
30     db.collection ("messages").document(titletext).set(message) //Add document to messages collection
31 }

```

Figure 5.20: Code from *ReviewMessageFragment* used to upload a new message to the messages collection.

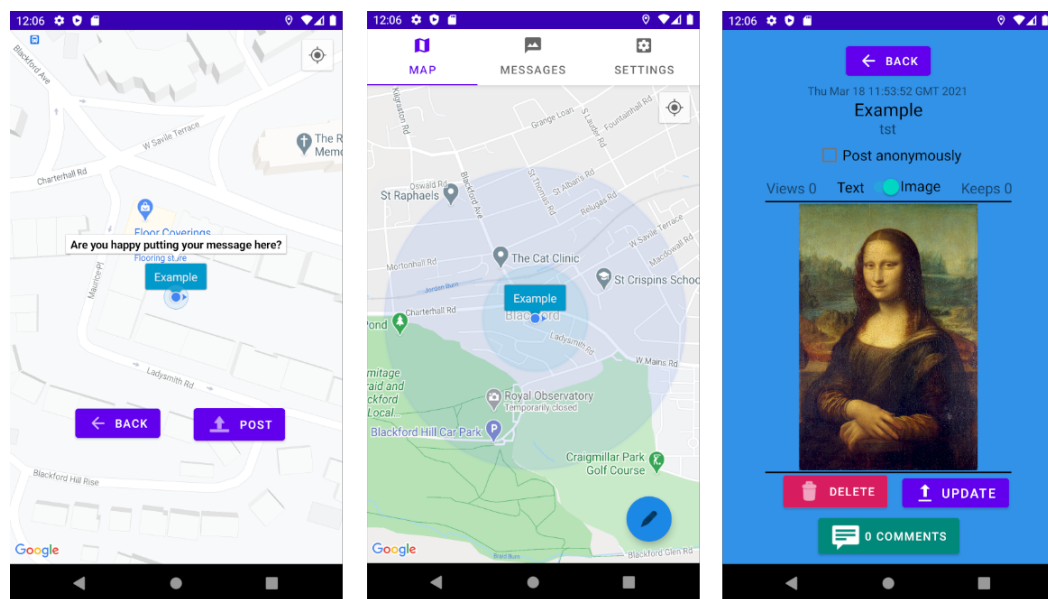


Figure 5.21: A message ready to post in *ReviewMessageFragment* (left), that message after posting on *MapMessageFragment* (middle), and *EditMessageFragment* when the author views that message (right).

deleted.

- The `messages` variable stores a mapping of a message's title to a `Message` object (`HashMap<String, Message>`), only messages owned by other users are stored here.
- The `our_messages` variable stores a mapping of a message's title to a `Message` object (`HashMap<String, Message>`), it contains messages the current user has posted.
- The `markermessages` variable maps each marker in `markers` to a message in `messages`, storing a (marker ID, message title) pair in a bidirectional hash map (`HashBiMap<String, String>`). This ensures that when a message is removed from a `HashMap`, its corresponding marker is also removed - i.e. when a message is deleted from the database it is also removed from the map.

If the message's author ID matches the user's ID (user currently logged in), the message is added to the `our_messages` `HashMap` and a corresponding blue marker is created using `createMarker()` (Figure 5.22). Otherwise it is another user's message so is added to the `messages` `HashMap` and depending on the state of the message (i.e. value/presence of `seen` field in `seenmessages`) the marker is displayed differently in terms of colour and title (section 4.4). Once all the nearby messages have been stored locally in a `HashMap` and markers have been created, both the `messages` and `our_messages` `HashMaps` are iterated to check for dead messages - i.e. messages deleted from the database must also be removed from the user's map.

```

1 | enum class STATES{UNSEEN, PARTIAL, SEEN, KEPT, OWN} //Enum class that enforces a message to be one of 5 states
2 |
3 | private fun createMarker(position: LatLng, title : String = "New Marker", state: String = "UNSEEN") {
4 |     if (markermessages.containsValue(title)) { //if the marker already exists
5 |         markers[markermessages.inverse()[title]]?.remove() //remove the marker from markers
6 |         markermessages.inverse().remove(title) //remove the marker from markermessages
7 |     }
8 |     val mIconGenerator = IconGenerator(this.requireContext()) //generate icon
9 |     when (STATES.valueOf(state)) {
10 |         OWN -> mIconGenerator.setStyle(IconGenerator.STYLE.BLUE) //if user's own message icon is blue
11 |         PARTIAL -> mIconGenerator.setStyle(IconGenerator.STYLE.ORANGE) //if user kept message icon is orange
12 |         else -> mIconGenerator.setStyle(IconGenerator.STYLE.GREEN) //otherwise message is green
13 |     }
14 |     var title2 : String = when (state) { //depending on the message's state we need to obscure title
15 |         UNSEEN -> "?".repeat(title.length) //unseen, hide all of title
16 |         PARTIAL -> "?".repeat(title.length/2)+title.drop( title .length/2) //partially seen, show half of title
17 |         else -> title
18 |     }
19 |     val iconBitmap: Bitmap = mIconGenerator.makeIcon(title2) //make icon
20 |     val marker = mMap.addMarker(MarkerOptions().position(position).icon(
21 |         BitmapDescriptorFactory.fromBitmap(iconBitmap))) //add the marker at the position with the title and icon
22 |     markers[marker.id] = marker //add the marker to list of markers
23 |     markermessages[marker.id] = title //add the marker-message pair
24 |     messages_states[title] = state //add the message's state
25 | }

```

Figure 5.22: Code from *MapFragment* used to create different markers on the map.

5.5.2 Viewing Messages via Markers

The primary way a user views a message and its contents is by tapping/clicking the marker on the map. If a message's state is only unseen or partially seen, then the user must get closer to view the message. Once a message is seen (i.e. `state = SEEN` or `seen = true` in `seenmessages`) the message can be viewed in `ViewMessageFragment` (Figure 5.23). However, if the user owns the message they will instead see `EditMessageFragment` which allows them to update or delete the message (Figure 5.21).

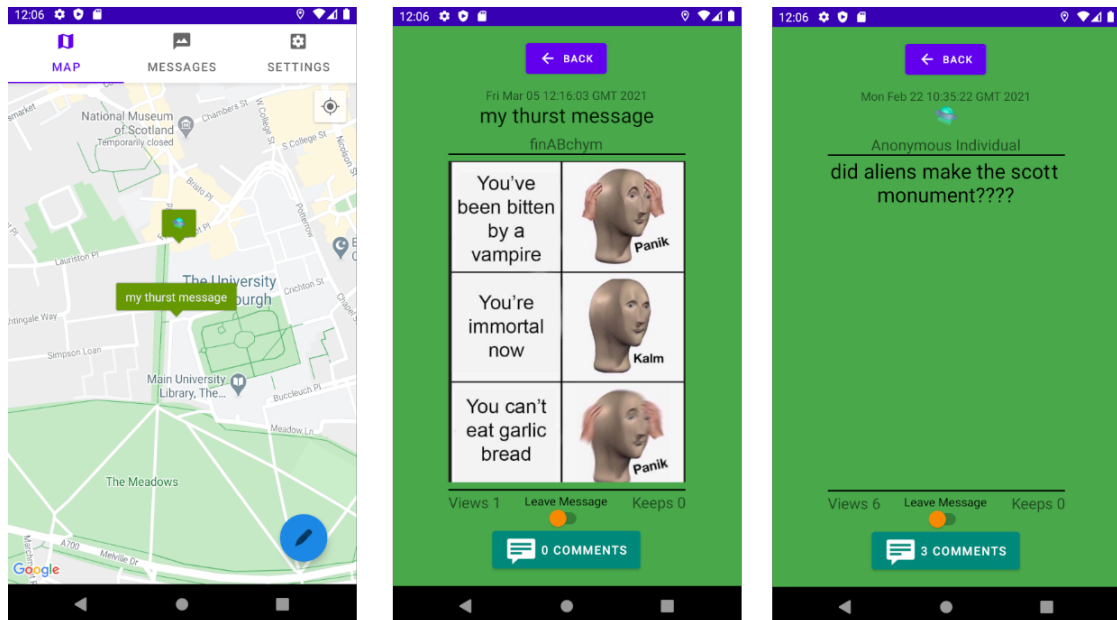


Figure 5.23: Two seen messages in `MapFragment` (left) both viewed in `ViewFragment`, one being an image message (middle), the other being a text message (right).

5.5.3 EditMessageFragment

As well as displaying messages differently to their author in terms of colour, authors also interact with their messages differently to other viewers (Figure 5.21). Firstly, authors have the ability to make changes to their message, i.e. anonymity and message's text/image. When the changes have been made they select update and the changes are made to the database via the `update()` method.[45] For example, to convert a message's content from an image to text the image field is set to empty and the text field is updated (Figure 5.7). Similarly, here authors have the ability to delete their messages which is done using `delete` (Figure 5.7).

5.6 Exploration and Proximity

In section 3.3, users are forced to move close to messages in order to interact with them, motivating them to move around the map (section 2.2.4). Furthermore, section 4.4 explains that messages can be in one of five states, which have a corresponding value in the `STATES` enum class (Figure 5.22):

- UNSEEN - the user is too far away so the message cannot be viewed, which is displayed as a green marker with the title hidden (replaced with question marks).
- PARTIAL - the user is too far away to view message, but close enough to now display a half-revealed title on the marker.
- SEEN - the user is close enough to view the message and the title is fully revealed.
- KEPT - The user has viewed the message and decided to keep it, so it is displayed as orange instead of green.
- OWN - The user created this message so can view/edit it at any time, which is displayed as blue.

In addition to the four HashMaps used to track messages and their markers (section 5.5.1), an additional HashMap `messages_states` is used to track the user's state to each message. It maps a message's title to the enum's state (HashMap<String, String>), e.g. a ("Holiday", UNSEEN) pair indicates that the user is too far away to view the message with the title "Holiday".

This mapping is created when markers are added to the map during `seenmessages` being checked (Figure 5.22). However, a message's state needs to be updated whenever a message is close enough to the user to be decoded (partially-seen) or seen. Users need a way to clearly see these two ranges, otherwise they will not understand why the titles of messages are changing. Hence these ranges are indicated by two circles (Figures 3.3 and 5.24). Every time a user's location is updated, old circles are removed and two new circles are placed on the map.[59]

Next, the user's proximity to every unseen/partially-seen message is checked every time a user's location is updated. In this case distance is checked twice - first to see if the message is in decode range (500m) and then to see if the message is in viewing range (175m). The range of 500m was chosen to be wide enough for some messages in a small area to be decoded, but not all. The range of 175m forces users to get very close to the message in order to view it, but does not force them to be at the exact coordinates.

Initially distance between the user and a message was calculated using Euclidean distance,[60] however as the earth is spherical and not on a 2D plane, this caused errors when calculating longitude. Instead, I used the haversine formula to determine the distance between two points[61], adapting my function (Figure 5.25) from another coded implementation.[62] If the message is less than 175m from the user, message's state (and `seen`) will be stored as true, otherwise it will be stored as false (section 5.3.3). If a message is seen by a user for the first time, that message's view count is incremented along with other statistics.

5.7 Comments

Rather than using a ScrollView which requires lists to be of a fixed size to display, a RecyclerView was used to display comments as they were dynamically added or removed (Figure 5.27).[63]

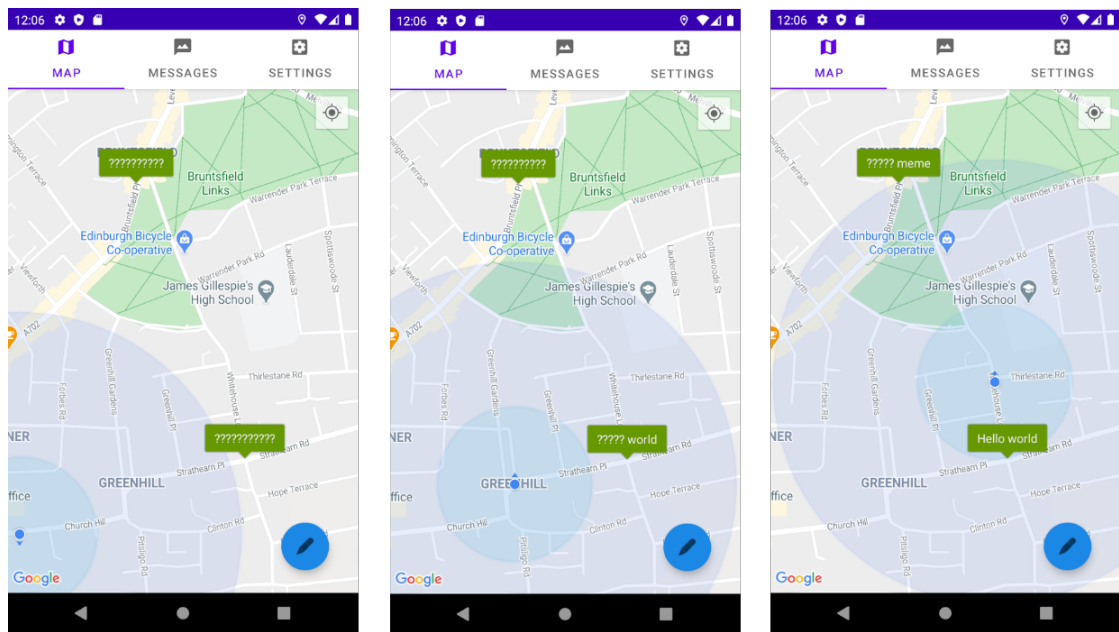


Figure 5.24: Map when two messages are unseen and out of range (left), map when one message is in decode range and the other is unseen (middle), map when one message is in seeing range and the other is decoded (right).

```

1 fun checkDistance(messageLoc :LatLng, currentLoc :LatLng, maxDistance : Double): Boolean {
2     return maxDistance > haversineDistance(messageLoc,currentLoc)
3 }
4
5 private val radian_converter: Double = (Math.PI/180)
6 fun haversineDistance(messageLoc :LatLng, currentLoc :LatLng): Double {
7     val a = sin ((( messageLoc.latitude—currentLoc.latitude) * radian_converter)/2)
8         * sin ((( messageLoc.latitude—currentLoc.latitude) * radian_converter)/2)
9         + cos((messageLoc.latitude)* radian_converter)
10        * cos((currentLoc.latitude)* radian_converter)
11        * sin ((messageLoc.longitude—currentLoc.longitude) * radian_converter/2)
12        * sin ((messageLoc.longitude—currentLoc.longitude) * radian_converter/2)
13    return 6371 * (2 * atan2(sqrt(a), sqrt(1—a)))
14 }

```

Figure 5.25: Code used to used to calculate distance using the haversine formula.

Initially comments were intended to appear below messages (Figure 3.1), with users being able to scroll down and interact with them, but `ViewFragment`'s available screen space was too limited to place a `RecyclerView`. To solve this, I attempted to use a `NestedScrollView`[64]. Within this a `CommentFragment`'s `RecyclerView` would sit below the static Views (i.e. title, image etc), which would be scrolled off-screen when users could scroll down the `RecyclerView`'s list of comments. However, the `RecyclerView` did not scroll correctly and I did not get it to work. As a compromise, I instead added a comments button which would swap in a `CommentsFragment` where users could better interact and view comments (Figure 5.26).

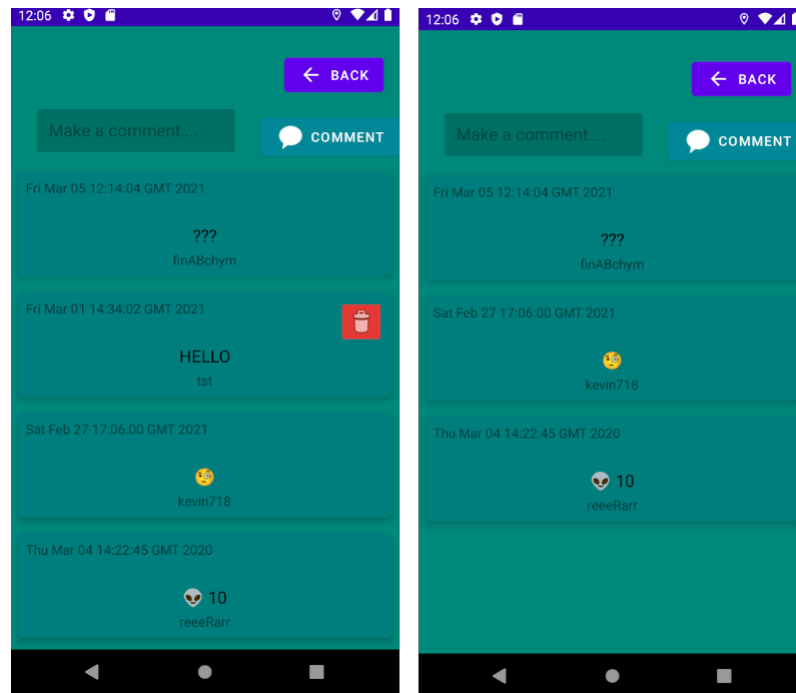


Figure 5.26: A *CommentFragment* showing a list of all comments (left) before a user deletes their comment (right).

```

1 private var comments.list : ArrayList<Comment> = ArrayList<Comment>()
2 val adapter = CommentAdapter(comments.list, this)
3 comments_recycler.adapter = adapter
4
5 db.collection("messages").document(title).collection("comments")
6 .addSnapshotListener { value, e -> //listen for changes in message's comments
7   if (e != null) { //return on any exceptions
8     Log.w(TAG, "Listen failed.", e)
9     return@addSnapshotListener
10  }
11  comments.list.clear() //reset the comments in list, as we will re-add them later
12  for (doc in value!!) { //for every comment document
13    comments.list.add(Comment( //add document to list of comments to show
14      id = doc.data["id"].toString(),
15      author_id = doc.data["author_id"].toString(),
16      time = doc.data["time"].toString(),
17      text = doc.data["text"].toString()))
18  }
19  adapter.notifyDataSetChanged() //update recyclerview with new list
20 }

```

Figure 5.27: Code from *CommentFragment* used to display a message's comments.

5.8 MessagesFragment

Similar to *CommentsFragment*, *MessagesFragment* uses a *RecyclerView* to display a list of messages dynamically, but now users have the ability to show or hide different types of messages. Furthermore, selected messages expand and allow users to view the message from the list (Figures 3.2 and 5.28).

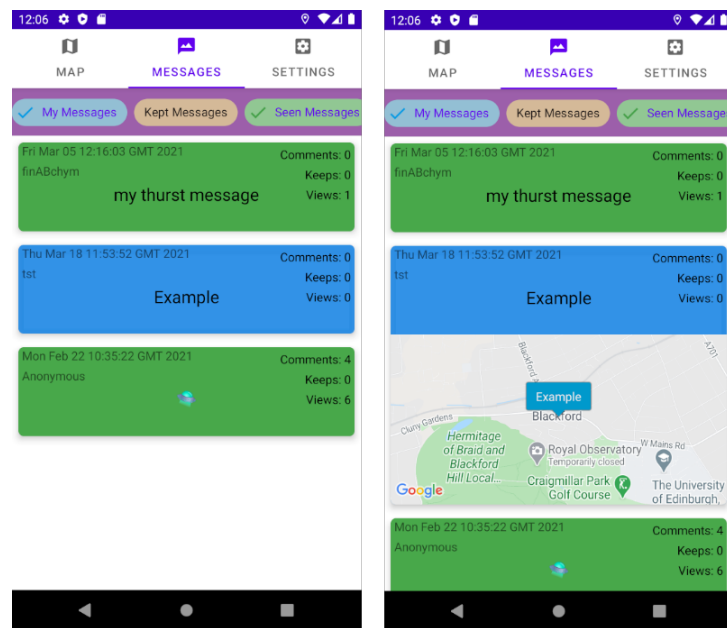


Figure 5.28: MessagesFragment showing that the user has seen two messages and has a message of their own (left), which can be viewed from a map marker (right).

5.9 Kept Messages

Instead of creating a new fragment for when a user keeps a message, a “kept fragment” is a ViewFragment (Figure 5.29). This allows users to easily “keep” or save messages for later (section 3.2).

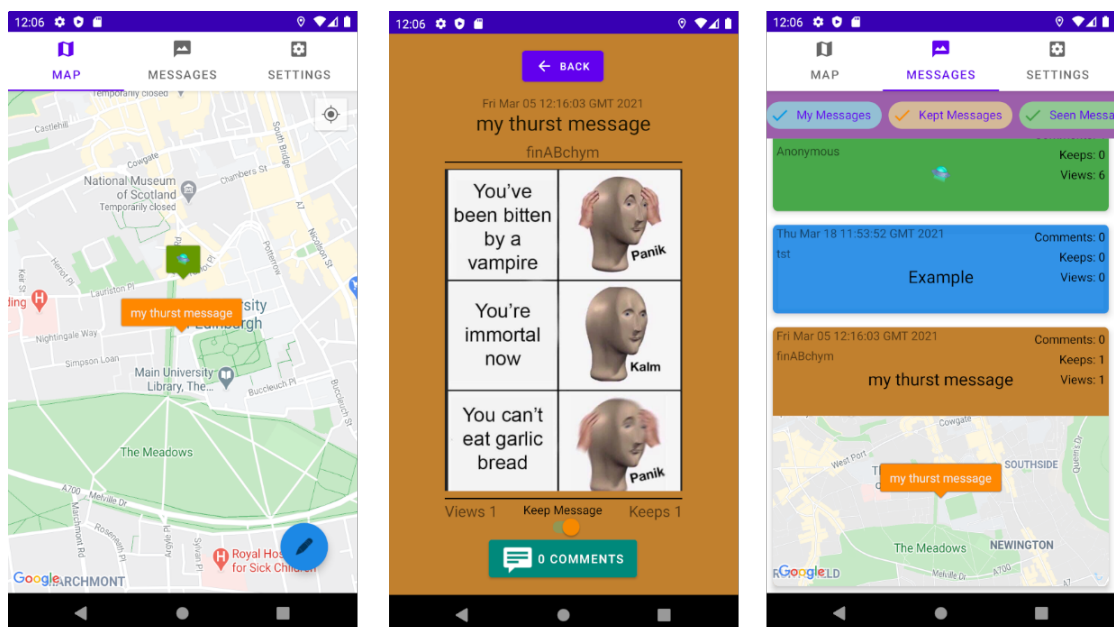


Figure 5.29: A kept message on MapFragment (left), in ViewMessageFragment (middle) and in MessagesFragment (right).

5.10 SettingsFragment

SettingsFragment is where the user views statistics, can log out or adjust what objects appear on their map (Figure 5.30), as specified in section 4.5.2.

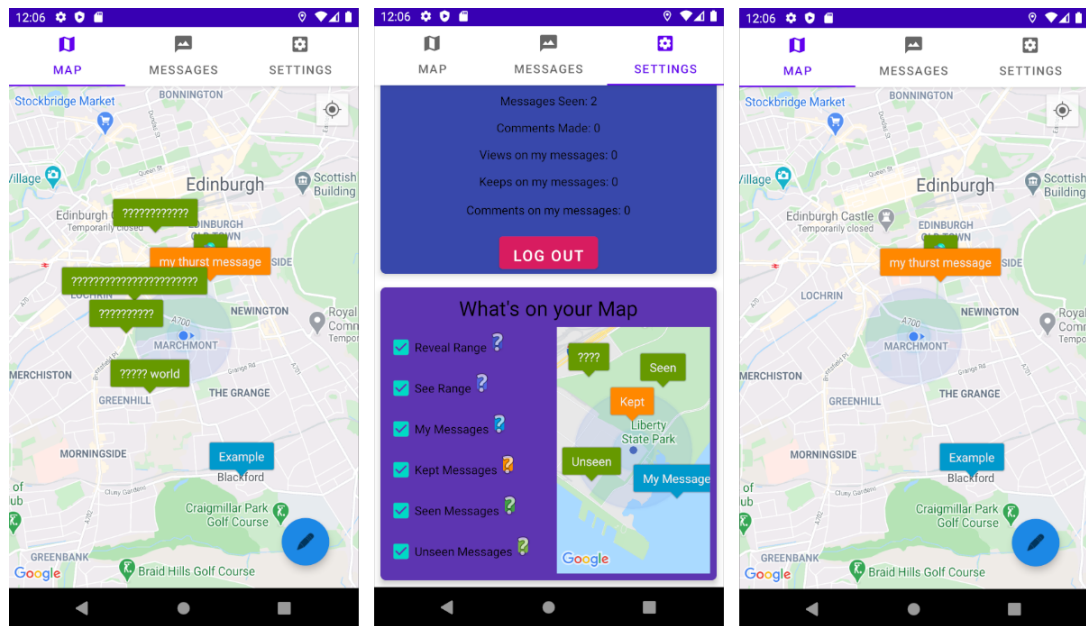


Figure 5.30: A cluttered *MapFragment* (left), *SettingsFragment* where the user can filter out different objects (middle) and a *MapFragment* where unseen messages have been filtered out (right).

Chapter 6

Evaluation

6.1 Research Questions

In order to determine if the fundamental design requirements of section 3.1 are met, Litter was tested on three questions:

1. *How usable is Litter?*
2. *How well does Litter do social-media?*
3. *Do users like Litter's location-based features?*

6.1.1 Usability

The first research question is used to assess how usable Litter is, i.e. do users have any difficulty in using Litter? A major aspect of this is the information shown to the user: do they understand what it means or find it confusing? Is current information insufficient or excessive? For example, do users understand that the pencil icon on the blue button means “create message” (Figure 5.23) or is this unintuitive? Another part of this specifically regards the layout and the UI - the *way* information is shown is just as important as *what* information is shown. For example, if the user has trouble seeing the back button within a kept message (Figure 5.29) it might as well not be there!

While there are several methods to measure usability, all involve users interacting with Litter. As covered in section 6.2.1, I decided that the best way would be to observe users performing tasks which would allow me to see any problems, i.e. where do users struggle to complete tasks?

6.1.2 Social-Media

The second research question is designed to examine Litter as a social media, i.e. do users understand Litter's social media features? Is communicating and interacting with others intuitive? While no doubt linked to how usable Litter is, here I am particularly interested in seeing users' thoughts on certain features. For example with “keeping” messages, do users understand it as a social feature similar to “liking/favouriting” a

message? Similarly, the intuitiveness of social-features is seen by watching users complete tasks. However, after users complete (or fail to complete) a task these features are discussed, allowing the users experience and actions to be explained clearly. Some features (such as “keeping”) are also assessed in a questionnaire, which provides a way to better compare users’ opinions.

6.1.3 Location-based features

The third research question examines Litter as a location-based app, i.e. what do users think of Litter’s location-based features? Especially important to discover is how intuitive these features are and whether or not users like them - i.e. exploration, do users like travelling to messages before viewing them? This is assessed by observing and discussing users’ interactions with these features, as well as in the questionnaire.

6.2 Methodology

There are many different methods on how to record and gather test data on Litter from users. One idea was to get users to install Litter, give them a list of tasks/tests to perform and allow them to report back their experiences in an interview or questionnaire. There are several concerns with this method. Firstly testers may not report back all problems they faced, e.g. they may consider a problem too minor to mention. Although this may be mitigated by a thoroughly extensive interview or questionnaire, this does not guarantee the issue is discovered - e.g. what if the problem is not covered in the questionnaire and the tester forgot to report it?

Secondly, hearing a tester’s experience via an interview or reading in a questionnaire has another issue - the lack of seeing exactly what the tester did. Not only must we assume the tester reported all details, but also that their report was effective and accurate enough for me to understand their experience correctly! For example, a tester may refer to parts of the interface differently from me, which I could misinterpret. Another example would be memory - can testers remember enough of their initial experience in an interview later? Without directly witnessing testers it’s difficult to understand how they completed tasks and why they did it that way and not another.

Therefore, I decided to instead use a Think-Aloud protocol[65], where users explain their thoughts as they interact with Litter and complete tasks. Meanwhile, the observer takes notes or records the tester. Ideally, this would be done in a “live test”, where the participant would be interacting with Litter and moving around in the real world, while the observer (myself) accompanied them, taking notes.

However, due to COVID-19 this was not feasible, so I instead decided to perform Think-Aloud using a “human computer” method.[66][67] In this method I share my screen and interact with Litter, with testers telling me what to click on and where to go. This would allow me to take notes, observe the participant’s responses and raise any queries I had. This also avoids requiring testers to download and install Litter onto their phones, allowing more testing time to be devoted to tasks and less to setup. This

was done by screen sharing Litter running on Android Studio's emulator in a Microsoft Teams call with the participant.[68]

6.2.1 Tasks

Each participant was given four tasks to complete:

1. *Login* - used primarily as a way to familiarise the participant to how they would test Litter, i.e. a tutorial or practice of directing me what to click on.
2. *View a message* - designed to cover research question 3 (Do users like Litter's location-based features?). In theory participants would see the map and its markers, try to view one, see that they need to move closer before trying again. This ensures the concept of exploration, a location-based aspect, is evaluated.
3. *Comment on a message* - designed to cover research question 2 (How well does Litter do social-media?), particularly seeing if they understood how commenting worked.
4. *Create a message* - designed to cover research question 3, seeing how participants would attempt to share their own message.

All of these tasks were essential for spotting usability issues (How usable is Litter?) and other problems the testers had with Litter. Once a user completed a task, I asked follow-up questions, clarifying what they thought and were trying to do, as well as checking if there were any issues they failed to mention.

6.2.2 Discussion

Four aspects of Litter were not covered by a task:

- Keeping Messages
- Statistics
- Filters
- All Messages

One reason these features were not made into tasks was because they were difficult to turn into useful tasks. For example, giving the task *View all messages in a list* would show me where participants thought this list was located, but this has little use beyond seeing any usability issues. Another reason was that these features provide user convenience and are not essential aspects of Litter, i.e. they can be removed or altered without affecting the main user experiences of exploration and communication. Finally, there are better ways to evaluate these features, i.e. discussing them with the participant.

Once a participant completed the tasks, I showed them these features and asked them what they thought they meant and their opinion of the feature. For example, I asked participants what they thought the “Keep message” switch (Figure 5.29) did before

going on to show them and hear their thoughts on it. This ensured that qualitative data on all of Litter's features was collected.

6.2.3 Questionnaire

In order to provide more comparable data between participants' experiences, they were asked five questions:

- On a scale of 1 to 5, where 1 was impossible and 5 was simple, how easy was it to:
 1. *View a message?*
 2. *Comment on a message?*
 3. *Create a message?*
- On a scale of 1 to 5, where 1 is hate and 5 is love, how much do you like:
 5. *Keeping messages?*
 6. *Discovering messages over time instead of immediately viewing them?*

The first three questions correspond to tasks 2-4, allowing for clear comparisons and analysis of usability between participants. Question 4 (*how much do you like keeping messages?*) provides data on keeping messages, a social feature not covered by any task. Question 5 (*how much do you like discovering messages over time instead of immediately viewing them?*) provides a direct way of gauging how participants felt about exploration, and Litter's approach to location in general.

6.2.4 Participants

Each participant had different experience using location-based and social media apps. Some regularly used location apps such as Pokemon Go, whereas others had little to no experience using these apps. This enabled me to see what both veterans and amateurs of location-centric apps thought of Litter's location-based features. All of my participants were familiar with social media, but others used it more frequently than others, providing an insight into how users considered Litter compared to other social-media.

Although a larger pool of participants provides more data, it has been demonstrated that after 5 participants it is unlikely further testers will find undiscovered problems.[69][70][71] Hence, Litter was tested on 5 participants to ensure a majority of problems were found.

6.3 Results

6.3.1 Viewing Messages

Overall, participants completed the task with few issues, however it did take a while for each to find out what they needed to do. One participant found the markers or

	Question	P1	P2	P3	P4	P5	Average
1	How easy was it to view a message?	5	4	3	5	3	4
2	How easy was it to comment?	5	5	4	5	5	4.8
3	How easy was it to create a message?	5	5	5	4	5	4.8
4	How much do you like “keeping” messages?	4.5	4	3	3	3	3.5
5	How much do you like discovering messages?	5	3	2	4	4	3.6

Figure 6.1: Questionnaire results where each participant gave a score out of 5: for questions 1-3 a 5 corresponds to “easy” and a 1 to “hard”, whereas for questions 4 and 5 a 5 corresponds to “love” and a 1 to “hate”.

“boxes” an unintuitive way of representing messages, another participant also said that it was “not obvious” the question marks represented unknown messages. One said it was difficult to understand the concept of range and another found it hard to distinguish between seen, partially-seen and unseen messages as they were all the same shade of green. One participant was initially confused by the popup when an unseen message was clicked and instead thought that “Too far away to read...” (Figure 6.2) implied they needed to zoom in to view the message. This was due to the vague wording not clearly directing the user, e.g. a better wording could be “You are too far away to view this message, move closer!” (Figure 6.2).

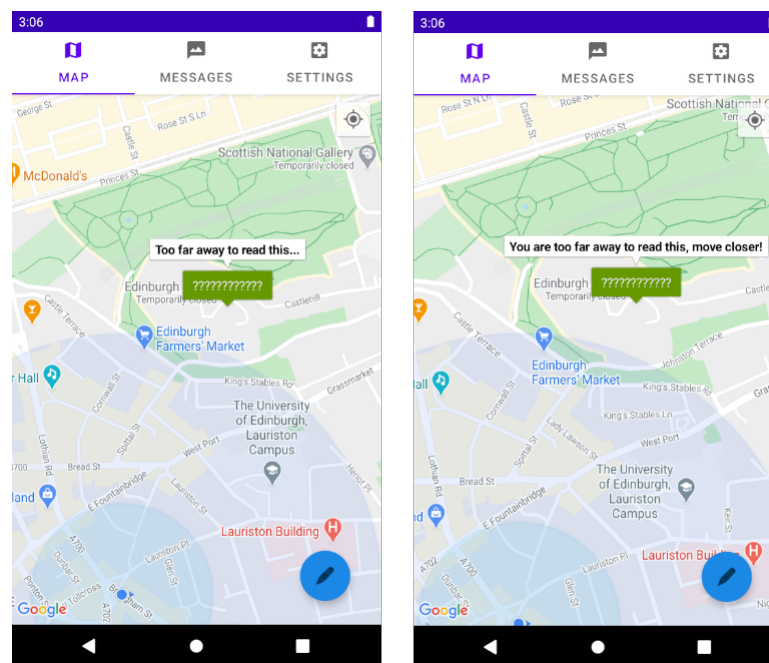


Figure 6.2: The popup that confused participant (left) was reworded to be clearer (right).

Several participants suggested that an additional in-app explanation was needed to help them understand what was happening: some suggested a quick pop-up giving a run-down on what to do and what was represented, others suggested that putting a tutorial message within decode range as an example would help.

6.3.2 Comments

Every participant easily located the “Comments” button and was able to make a comment. However, several participants remarked that they had trouble reading comments that the “black text was small and hard to read”, that it was “difficult to read black text on dark background” and that it was hard to distinguish between their comment and other comments. Several users suggested an immediate improvement would be changing the colour of the text to white or brightening the background colour (Figure 6.3).

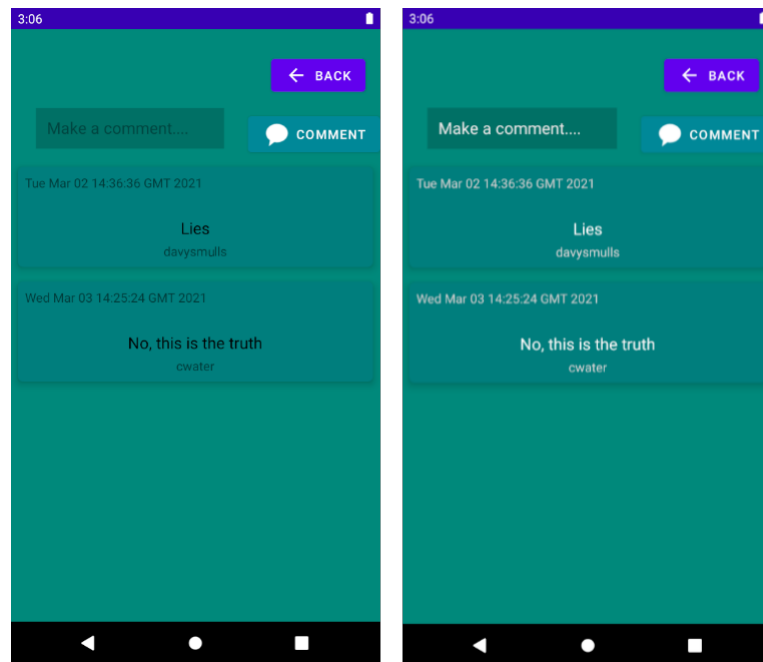


Figure 6.3: To improve readability the text in `CommentsFragment` was recoloured from black (left) to white (right).

Participants also expressed a desire for more ways to interact with comments, either by replying to a comment or commenting on a comment (sub-comment) or using reaction emojis (e.g. users can like or heart messages).

6.3.3 Creating Messages

Similarly, participants had little difficulty creating messages. One participant initially did go to the All Messages tab first and expected to be able to create a message there, but quickly returned to the map and found the correct button.

One participant was confused when reviewing the message before posting, particularly by the popup “do you wish to post your message here?” (Figure 6.4), which led them to believe that they could drag around the marker and adjust its location. After learning this was not possible, they suggested to remove “here” in the popup and changed the “Next” button to “Review” (Figure 6.4).

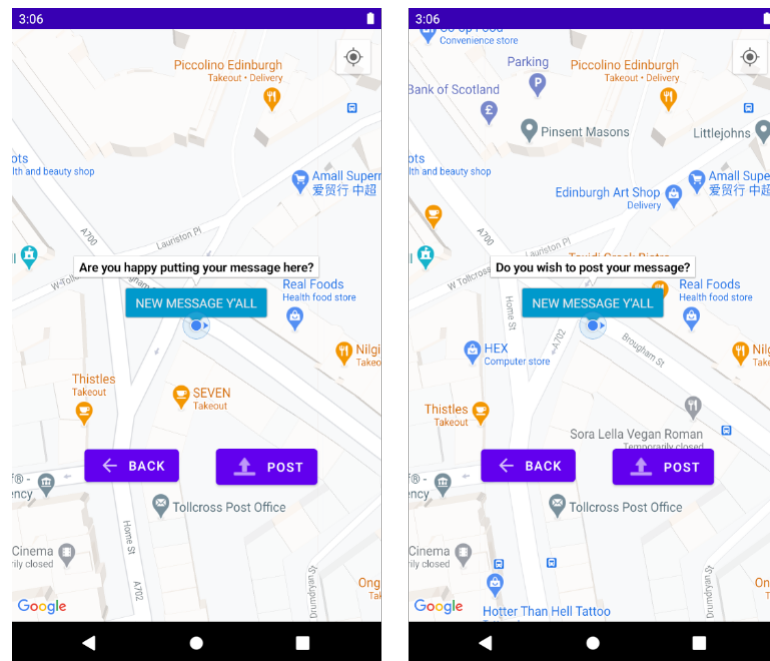


Figure 6.4: *The wording of the review popup (left) was updated to avoid confusion (right).*

6.3.4 Keeping Messages

No participant correctly understood or guessed what the “Leave Message” switch did and found the wording and use of the switch unclear. Several mentioned that they would prefer a button instead of switch as it is more intuitive to how messages are saved on other social medias. Some suggested that wording it “save” instead of “keep” would also have avoided confusion. Despite the usability issues around it, participants still strongly wanted a way to save messages for later and liked the idea.

6.3.5 Exploration

Generally participants liked the idea of exploration, one saying that hiding messages provided “good incentive to explore”. Some found it engaging but a time-consuming way to communicate (i.e. can’t immediately contact or get in touch with someone), whereas others were lukewarm to the idea, neither liking nor disliking it. One participant felt that it was not strict enough and that users should not be able to view seen messages when out of range, only being able to view kept messages from afar.

Several participants felt that the current system of partially-revealing titles was underwhelming and expressed a desire to see a more detailed preview as they got closer to the message, e.g. initially see a header or title, then a description or image.

6.3.6 Other Issues and Suggestions

Other usability issues were raised by participants. One participant was confused by the label of “message” as they associate “messages” with direct communication, whereas

a better description for Litter would be “broadcasts” or “posts”. Some participants found it difficult to understand what the list of messages represented in the Messages tab, or that they could click and interact with them. This further indicates that users struggle to understand the current way messages are portrayed, as seen similarly in task 2’s results (section 6.3.1).

One participant remarked that Litter is similar to SnapMap, a spinoff feature of SnapChat,^{[72][73]} which would be a useful point of comparison for any future work. One participant described Litter as providing location-based “message boards”, as users create a message where others comment and communicate with others. They suggested that authors should have the ability to lock or archive a message, and the ability to restrict access to certain groups or individuals. Additionally, several participants expressed a desire to restrict or direct messages towards groups or friends, rather than strictly broadcasting in public. One participant suggested either going for a follower-based system (i.e. Twitter) where users follow and respond to messages made by others, or a friend-based system where users more directly interact with others.

6.4 Recommended Fixes and Changes

Although I made minor changes based on user feedback (Figures 6.2-6.4), Litter needs several substantial updates addressing other issues raised by users. Firstly, participants struggled to link messages to markers on the map, initially confused where or how to interact with messages. As suggested by some testers, implementing a tutorial or introductory screen that explains to users what a message is and what it looks like would vastly help this problem. Some example messages should also be included to allow users to familiarise with interacting with messages in different states.

Secondly, messages in different states must be distinctly portrayed on the map and unseen messages specifically should not be the same shade of green. Additionally, participants were underwhelmed with how messages are decoded/revealed over time and a new system with a more interesting preview of the message would engage users better. Initially users might only see the title and type of message (i.e. image or text), but as the user gets closer they can see a preview of the message’s description or image.

Thirdly, “keeping” is too vague a concept for users to intuitively understand and should be relabeled. Furthermore the use of a switch confused testers and it should be replaced with a button. The concept should either be relabelled to “saving” or “following” which users should better understand.

Finally, testers wanted additional ways to be social towards others. Primarily, a friend system would provide this. Here, users could mention an individual or group in a message or comment, notifying them to travel to and view a message. This could also be done by giving authors the ability to hide or restrict the visibility of messages to a group of individuals, allowing users to communicate more directly. Furthermore, implementing a similar follower system where popular individuals can notify their followers when a new message is broadcast would. This could also serve as a replacement to the concept of “keeping”.

Chapter 7

Conclusion

7.1 Discussion of Achievements

I successfully achieved the goal of creating a location-based social media android app. Unlike Blummi or EagleTweet, Litter is a functional Android app. Unlike Swarm or SKOUT, Litter is fundamentally a social media while also being location-based.

Litter allows users to create and interact with others by moving around and exploring. Testing showed that participants enjoyed the system of exploring, inspired by Pokemon GO and Geocaching.

Although participants expressed a desire for more options on how to be social (e.g. directing their interactions towards friends), participants approved of the current social features of messages, comments and keeping despite usability issues.

Results, such as the questionnaire (Figure 6.1), showed that overall participants found Litter usable and intuitive.

In conclusion, Litter meets the requirements of a location-based social media, providing a solid foundation and example for future projects.

7.2 Future Work

While any future work should prioritise the fixes and suggestions outlined in section 6.4 (e.g. friends and follower systems), there are several additional features that would augment Litter as a location-based social media:

- A system of grouping related messages together via keywords (similar to Twitter's Hashtags) which users can then use to filter out or specify what content/topics they wish to view.
- Various notifications which alert the user of activity in Litter outside of the app. For example, users are alerted if their message is commented on or if they are nearby a message. These would remind users of events happening within Litter.

- A compass feature that shows users which direction a message is. Similarly, a button that pans/zooms to the nearest unseen message would simplify exploration on a cluttered map. These would help guide users towards messages.
- Account integration with other services (e.g. Facebook or Google) would speed up the registration and login process. Furthermore, user's emails are not currently verified which may lead to exploitation (i.e. easier for bot/fake account creation) or errors (what if a user entered an incorrect email?).
- The ability to reply to a specific comment would help users communicate with others directly.
- The ability to react to a comment using an emoji (instead of text) provides a quick and simple way for users to communicate how they feel about a comment.

7.3 Future Testing

As well as testing extensions and fixes to Litter, there are several areas which need covered in future evaluations:

- Testing and evaluation was performed using Android Studio's emulator, which although reliable is arguably not completely representative of users' devices. In future testing should be done on a variety of physical mobiles with a variety of android versions. This testing should include performance and latency tests, e.g. time taken to render map/message.
- Evaluation was done one user at a time, independently of other users. In reality Litter is a social media and multiple users can interact simultaneously - this should be reflected in future tests.
- A major justification for the list of messages in `MessagesFragment` was to provide an alternate way to access messages. However, users never encountered a situation where they had difficulty accessing messages, i.e. two messages close together. In future not only should users' responses to this situation be recorded, but their feedback on how it should be resolved. It would be interesting to hear user thoughts on: clustering (displaying a cluster instead of multiple markers), competitions (users vote for which message should occupy the space), overlaying unseen messages on top of others or first-come-first-served (first message posted permanently occupies a radius around a location) as solutions.
- In future a survey on the label of "posts" as a better description of what are currently "messages" should be performed.

Bibliography

- [1] Driving directions, traffic reports & carpool rideshares by waze. <https://www.waze.com/en-GB/>.
- [2] Spot adventures. <https://spotadventures.com>.
- [3] Foursquare city guide. <https://foursquare.com/city-guide>.
- [4] Blummi! <https://web.archive.org/web/20110902142852/http://www.blummi.com/>. Accessed via Wayback Machine.
- [5] Eagletweet. <http://eagletweet.com>.
- [6] Twinkle. <https://web.archive.org/web/20090803043632/http://tapulous.com/twinkle/>. Accessed via Wayback Machine.
- [7] Twibble. <https://web.archive.org/web/20090701104144/http://www.twibble.de/>. Accessed via Wayback Machine.
- [8] Jemima Kiss. Yahoo launches fire eagle location tool. <https://www.theguardian.com/media/2008/aug/13/yahoo.digitalmedia>, 2008.
- [9] Swarm - remember everywhere. https://www.swarmapp.com/?_branch_match_id=841248622390227021.
- [10] What is foursquare swarm? <https://support.foursquare.com/hc/en-us/articles/201908440-What-is-Foursquare-Swarm->.
- [11] Yelp. <https://www.yelp.co.uk/>.
- [12] Skout. <https://www.skout.com>.
- [13] Kim-Mai Cutler. Skout launches fuse, an ephemeral messaging app that makes group chats explode. <https://techcrunch.com/2014/06/19/andreessen-fuse/>, 2014.
- [14] How do i add my location to an existing facebook post? <https://www.facebook.com/help/115298751894487>.
- [15] How to add your location to a tweet. <https://help.twitter.com/en/using-twitter/tweet-location>.
- [16] How to use live location. <https://faq.whatsapp.com/android/chats/how-to-use-live-location/?lang=en>.

- [17] Geocaching. <https://www.geocaching.com/play>.
- [18] Geocaching mobile apps. <https://www.geocaching.com/play/mobile>.
- [19] c:geo. https://play.google.com/store/apps/details?id=cgeo.geocaching&hl=en_US.
- [20] Ingress prime. <https://www.ingress.com>.
- [21] Edmond Tran. Pokemon go devs still exploring multiplayer, say players will shape the game. <https://www.gamespot.com/articles/pokemon-go-devs-still-exploring-multiplayer-say-pl/1100-6441610/>, 2016.
- [22] Pokémon go. <https://www.pokemongo.com/en-us/>.
- [23] Nick Statt. Google's april fools' prank puts pokemon in the real world. <https://www.cnet.com/news/googles-april-fools-prank-puts-pokemon-in-the-real-world/>, 2014.
- [24] Conor Duffy. What is social media phenomenon pokemon go? <https://www.abc.net.au/news/2016-07-11/what-is-pokemon-go/7587346>, 2016.
- [25] Erik Kain. Pokémon go is more than just a game and it's bringing people together. <https://www.forbes.com/sites/erikkain/2016/07/11/pokemon-go-is-more-than-just-a-game-its-a-phenomenon-thats-bringing-people-together/#5f4df9864743>, 2016.
- [26] Get started with the ndk. <https://developer.android.com/ndk/guides>.
- [27] Android's kotlin-first approach. <https://developer.android.com/kotlin/first>.
- [28] Twitter increases developer productivity and code reliability with kotlin. <https://developer.android.com/stories/apps/twitter-kotlin>.
- [29] Download android studio and sdk tools. <https://developer.android.com/studio/>.
- [30] Sqlite. <https://www.sqlite.org/index.html>.
- [31] Save data using sqlite. <https://developer.android.com/training/data-storage/sqlite>.
- [32] Save data in a local database using room. <https://developer.android.com/training/data-storage/room>.
- [33] MongoDB, the database for modern applications. <https://www.mongodb.com/>.
- [34] Cloud firestore. <https://firebase.google.com/products/firestore>.
- [35] Comparing firebase vs mongodb. <https://www.mongodb.com/firebase-vs-mongodb>.
- [36] Mapbox. <https://www.mapbox.com>.

- [37] Geo-location apis - google maps platform. <https://cloud.google.com/maps-platform>.
- [38] Introduction to activities. <https://developer.android.com/guide/components/activities/intro-activities>.
- [39] Fragments. <https://developer.android.com/guide/fragments>.
- [40] Firebase authentication. <https://firebase.google.com/docs/auth>.
- [41] Security rules and firebase authentication. <https://firebase.google.com/docs/rules/rules-and-auth#cloud-firestore>.
- [42] Authenticate with firebase using password-based accounts on android. <https://firebase.google.com/docs/auth/android/password-auth>.
- [43] Fragment transactions. <https://developer.android.com/guide/fragments/transactions>.
- [44] Get started with cloud firestore. <https://firebase.google.com/docs/firestore/quickstart>.
- [45] Add data to cloud firestore. <https://firebase.google.com/docs/firestore/manage-data/add-data>.
- [46] Slide between fragments using viewPager. <https://developer.android.com/training/animation/screen-slide>.
- [47] Create swipe views with tabs using viewPager. <https://developer.android.com/guide/navigation/navigation-swipe-view>.
- [48] Map objects. https://developers.google.com/maps/documentation/android-sdk/map#add_map_code.
- [49] Request app permissions. <https://developer.android.com/training/permissions/requesting>.
- [50] Location data. https://developers.google.com/maps/documentation/android-sdk/location#request_runtime_permissions.
- [51] Request location permissions. <https://developer.android.com/training/location/permissions>.
- [52] Request location updates. <https://developer.android.com/training/location/request-updates>.
- [53] Get the last known location. <https://developer.android.com/training/location/retrieve-current>.
- [54] Cloud firestore data model. <https://firebase.google.com/docs/firestore/data-model>.
- [55] Get started with cloud storage on android. <https://firebase.google.com/docs/storage/android/start>.

- [56] Upload files with cloud storage on android. <https://firebase.google.com/docs/storage/android/upload-files>.
- [57] Google maps android api utility library. <http://googlemaps.github.io/android-maps-utils/>.
- [58] Maps sdk for android utility library. <https://github.com/googlemaps/android-maps-utils>.
- [59] Shapes — maps for sdk android — google developers. <https://developers.google.com/maps/documentation/android-sdk/shapes#circles>.
- [60] Euclidean distance. https://en.wikipedia.org/wiki/Euclidean_distance.
- [61] Haversine formula. https://en.wikipedia.org/wiki/Haversine_formula.
- [62] Calculate distance, bearing and more between latitude/longitude points. <https://www.movable-type.co.uk/scripts/latlong.html>.
- [63] Create dynamic lists with recyclerview. <https://developer.android.com/guide/topics/ui/layout/recyclerview>.
- [64] Nestedscrollview. <https://developer.android.com/reference/androidx/core/widget/NestedScrollView>.
- [65] Think aloud protocol. https://en.wikipedia.org/wiki/Think_aloud_protocol.
- [66] Dr Kami Vaniea. What are iterative design and fast feedback methods (slides). https://www.learn.ed.ac.uk/bbcswebdav/pid-5194193-dt-content-rid-16021341_1/xid-16021341_1, 2020.
- [67] Dr Kami Vaniea. What are iterative design and fast feedback methods lecture. https://media.ed.ac.uk/media/Fast+Feedback+and+Design+Iteration/1_lwe4jkd2, 2020.
- [68] Microsoft teams. <https://www.microsoft.com/en-gb/microsoft-teams/group-chat-software>.
- [69] Ellie Martin. Why 5 is the magic number for ux usability testing. <https://www.invisionapp.com/inside-design/ux-usability-research-testing/>, 2016.
- [70] Jakob Nielsen. Why you only need to test with 5 users. <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/>, 2000.
- [71] Usability testing. https://en.wikipedia.org/wiki/Usability_testing#Number_of_test_subjects.
- [72] About snap map. <https://support.snapchat.com/en-GB/a/snap-map-about>.
- [73] Snap map. <https://map.snapchat.com/>.